

Complete ML Engineering Handbook

Table of Contents

- Three End-to-End Projects for Production Deployment
- Table of Contents
- Executive Summary
 - Why These Projects Matter
- PROJECT 1: AI Fraud Detection System
 - 1.1 Executive Summary
 - 1.2 Project Architecture Diagram
 - 1.3 Technical Stack & Requirements
- Core dependencies
- Model management
- Explainability
- API & deployment
- Visualization
- Testing
- Utilities
 - 1.4 Dataset Guide
- Load data
- Basic exploration
- Check for missing values
- Check for missing values (Kaggle dataset is clean)
- Outlier detection (robust to fraud patterns)
- Use IQR for legitimate transactions only
- Keep transactions within reasonable range
- (don't remove - fraud might be in outliers!)
- TIME-BASED FEATURES
- AMOUNT-BASED FEATURES
- ROLLING FEATURES (aggregate by user behavior)
- Note: Original dataset doesn't have user ID, so we simulate
- VELOCITY FEATURES (transactions per hour)
- Separate features and target
- Standardize Amount (others are PCA-transformed)
- For is_night, is_small_amount, is_large_amount: already 0/1, no scaling needed
- For other engineered features: scale them
- Stratified split
- Apply SMOTE only to training data

- 1.5 Model Development
- Train baseline model
- Predict
- Evaluate
- XGBoost is the de facto standard for structured data
- Feature importance (why predictions matter)
- Unsupervised: useful for novel fraud patterns
- Train on all data (unsupervised)
- Create comparison table
- DECISION: XGBoost is production choice (best balance of metrics + explainability)
 - 1.6 Hyperparameter Tuning with Optuna
- Run study
- Best parameters
- Retrain with best parameters
 - 1.7 Model Evaluation & Metrics
- Interpretation:
- AUC = 0.95 means: 95% probability that model ranks a random fraud higher than random legitimate
- Better for imbalanced data! Shows tradeoff between TP and FP at different thresholds
- Business metrics
- Create SHAP explainer
- Summary plot: which features matter most?
- Force plot: why specific transaction was flagged as fraud?
- Dependency plot: how does Amount affect fraud probability?
 - 1.8 Model Registry with MLflow
- Set experiment
 - 1.9 Deployment: FastAPI Server
- app/main.py
- Load model
- Dockerfile
- Install dependencies
- Copy application
- Health check
- Run app
- docker-compose.yml
 - 1.10 Monitoring & Retraining
- monitoring/drift_detection.py
- Usage
- Monitor new batch
- monitoring/retraining_trigger.py
- dashboard/app.py

- [Sidebar filters](#)
- [Load metrics](#)
- [1. Key metrics](#)
- [2. Performance over time](#)
- [3. Fraud distribution](#)
- [By hour](#)
- [By amount](#)
- [4. Recent alerts](#)
 - [1.11 Interview Talking Points](#)
- [PROJECT 2: ML-Powered Recommendation System](#)
 - [2.1 Executive Summary](#)
 - [2.2 Recommendation System Architecture](#)
 - [2.3 Dataset Selection & Preparation](#)
- [Download](#)
- [Contents:](#)
- [- ratings.csv: userId, movieId, rating, timestamp](#)
- [- movies.csv: movieId, title, genres](#)
- [- tags.csv: userId, movieId, tag, timestamp](#)
- [Load data](#)
- [Sparsity analysis](#)
- [Rating distribution](#)
- [Power-law: most items have few ratings](#)
- [1. Filter cold-start items \(items with <5 ratings\)](#)
- [2. Filter cold-start users \(users with <5 ratings\)](#)
- [3. Create user-item matrix](#)
- [4. Train-test split \(temporal\)](#)
- [Use recent ratings for testing](#)
 - [2.4 Content-Based Filtering](#)
- [1. TF-IDF on movie titles](#)
- [2. Genre encoding \(multi-hot\)](#)
- [3. Combine features](#)
- [Normalize genre matrix](#)
- [Combine with title TF-IDF \(60% title, 40% genre\)](#)
- [Compute item-item similarity matrix](#)
- [Example: Find movies similar to "Inception"](#)
- [Test](#)
 - [2.5 Collaborative Filtering](#)
- [Convert to sparse matrix format \(implicit library uses user x item\)](#)
- [Train ALS model](#)
- [Test](#)

- [2.6 Hybrid Recommendation System](#)
- [Test](#)
 - [2.7 Evaluation Metrics](#)
- [Evaluate on test set](#)
- [Print results](#)
 - [2.8 FastAPI Deployment](#)
- [app/main.py](#)
- [Load models](#)
- [Load matrices](#)
 - [2.9 Interview Talking Points for Project 2](#)
- [PROJECT 3: Autonomous ML Operations Pipeline](#)
 - [3.1 Executive Summary](#)
 - [3.2 MLOps Architecture](#)
 - [3.3 Project Setup](#)
- [Create virtual environment](#)
- [Install dependencies](#)
- [Required packages](#)
- [Core dependencies](#)
- [MLOps](#)
- [Deployment](#)
- [Monitoring](#)
- [Testing](#)
- [Utils](#)
- [Initialize Airflow](#)
 - [3.4 Data Ingestion & Validation](#)
- [src/data_ingestion.py](#)
- [Usage in Airflow task](#)
- [src/data_validation.py](#)
- [Usage](#)
 - [3.5 Feature Store](#)
- [src/feature_store.py](#)
- [Usage in Airflow](#)
 - [3.6 Model Training Pipeline](#)
- [src/model_training.py](#)
- [Airflow DAG](#)
 - [3.7 Model Evaluation & Selection](#)
- [src/model_evaluation.py](#)
 - [3.8 Model Registry & Deployment](#)
- [src/model_registry.py](#)
- [Deployment with Docker](#)

- [API server](#)
- [Load model from registry](#)
 - [3.9 Monitoring & Alerting](#)
- [monitoring/prometheus_metrics.py](#)
- [Define metrics](#)
- [Middleware for metrics](#)
- [Drift detection task](#)
- [Grafana dashboard \(as code\)](#)
- [Alert rules](#)
 - [3.10 Automated Retraining Decision](#)
- [dags/automated_retraining.py](#)
 - [3.11 Interview Talking Points for Project 3](#)
- [System Diagrams & Architecture](#)
 - [Data Flow Diagram](#)
 - [Model Lifecycle Diagram](#)
- [Code Templates & Implementation](#)
 - [Complete Fraud Detection Training Pipeline](#)
- [pipelines/fraud_detection_complete.py](#)
- [Testing & Production Considerations](#)
 - [Unit Tests](#)
- [tests/test_models.py](#)
- [Interview Talking Points](#)
 - [General Project Opening](#)
 - [Technical Depth Questions](#)
- [Appendix](#)
 - [Resources](#)
 - [Useful Commands](#)
- [Build](#)
- [Run](#)
- [Push to registry](#)
- [Deploy](#)
- [Check status](#)
- [Scale](#)
- [Start server](#)
- [View UI](#)
- [Navigate to http://localhost:5000](#)
- [Initialize](#)
- [Create user](#)
- [Start scheduler](#)
- [Start webserver](#)

- [Common Errors & Solutions](#)

Three End-to-End Projects for Production Deployment

Table of Contents

1. [Executive Summary](#)
2. [Project 1: AI Fraud Detection System](#)
3. [Project 2: ML-Powered Recommendation System](#)
4. [Project 3: Autonomous ML Operations Pipeline](#)
5. [System Diagrams & Architecture](#)
6. [Code Templates & Implementation](#)
7. [Testing & Production Considerations](#)
8. [Interview Talking Points](#)
9. [Appendix](#)

Executive Summary

This handbook is a complete engineering roadmap for building three industry-grade machine learning systems that will demonstrate your ability to work in production environments.

Why These Projects Matter

Project 1: Fraud Detection demonstrates:

- Real-world imbalanced data handling
- Feature engineering and domain knowledge
- Risk-aware model evaluation (False Positives vs False Negatives)
- End-to-end system ownership

Project 2: Recommendation System demonstrates:

- Scalable data engineering
- Advanced ML mathematics (matrix factorization, embeddings)
- Hybrid system design
- Real business application (used at Netflix, Spotify, Amazon)

Project 3: MLOps Pipeline demonstrates:

- Production ML infrastructure thinking
- Automated workflows and orchestration
- Model monitoring and governance
- Career-critical MLOps skills

Together, these projects prove you can **build, deploy, monitor, and maintain ML systems at scale**.

PROJECT 1: AI Fraud Detection System

1.1 Executive Summary

Problem Statement

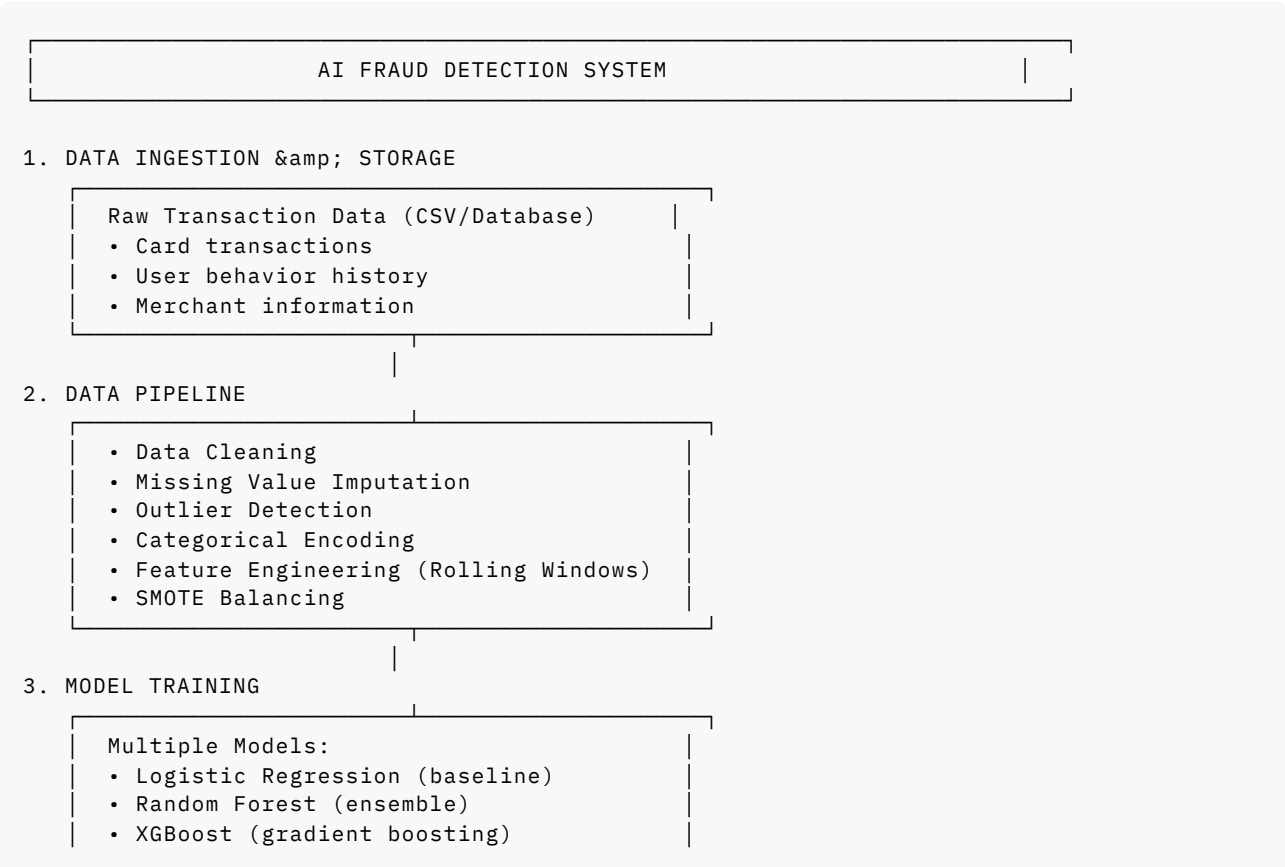
Fraud costs financial institutions **\$28.65 billion annually** (2021). Traditional rule-based systems generate excessive false positives, degrading user experience. Your system will:

- Identify fraudulent transactions in real-time with **90%+ recall** and **high precision**
- Handle **highly imbalanced data** (fraud rate ~0.17%)
- Provide **explainable predictions** using SHAP values
- Deploy as **production-ready API** with monitoring

Success Metrics

Metric	Target	Why
ROC-AUC	≥0.95	Comprehensive model quality
Precision	≥0.85	Minimize false positives (customer frustration)
Recall	≥0.90	Catch actual fraud (business loss prevention)
F1-Score	≥0.87	Balance precision-recall tradeoff
Latency	<100ms	Real-time transaction processing

1.2 Project Architecture Diagram





1.3 Technical Stack & Requirements

Core Technologies

Component	Technology	Why
Language	Python 3.10+	Industry standard for ML
Data Processing	Pandas, NumPy	Fast tabular data manipulation

Component	Technology	Why
Feature Engineering	Pandas, Scikit-learn	Scalable feature pipelines
Imbalance Handling	Imbalanced-learn (SMOTE)	Handle 0.17% fraud rate
Model Training	Scikit-learn, XGBoost	Proven, production-ready
Hyperparameter Tuning	Optuna	State-of-the-art Bayesian optimization
Model Registry	MLflow	Industry-standard model management
Explainability	SHAP	Model interpretability at scale
API Framework	FastAPI	Modern, async, automatic docs
Containerization	Docker	Reproducible deployments
Monitoring	Prometheus + Grafana	Production-grade monitoring
Dashboard	Streamlit	Rapid prototype + deploy
Orchestration	Airflow (optional)	Schedule retraining pipelines

Required Libraries

```
# Core dependencies<a></a>
pandas==2.1.0           # Data manipulation
numpy==1.24.0           # Numerical computing
scikit-learn==1.3.0     # Classic ML algorithms
xgboost==2.0.0          # Gradient boosting
imbalanced-learn==0.11.0 # SMOTE for imbalanced data
optuna==3.4.0           # Hyperparameter tuning

# Model management<a></a>
mlflow==2.10.0          # Model registry & tracking
joblib==1.3.0           # Model serialization

# Explainability<a></a>
shap==0.43.0            # SHAP values for interpretability

# API & deployment<a></a>
fastapi==0.105.0        # Async web framework
uvicorn==0.24.0         # ASGI server
pydantic==2.5.0         # Data validation

# Visualization<a></a>
streamlit==1.31.0       # Dashboard framework
matplotlib==3.8.0       # Static plots
seaborn==0.13.0         # Statistical plots
plotly==5.18.0          # Interactive charts

# Testing<a></a>
pytest==7.4.0           # Unit testing
pytest-cov==4.1.0       # Code coverage

# Utilities<a></a>
python-dotenv==1.0.0    # Environment variables
requests==2.31.0        # HTTP requests
```

1.4 Dataset Guide

Recommended Dataset

Primary: Credit Card Fraud Detection Dataset (Kaggle)

- **Size:** ~284,807 transactions
- **Fraud Rate:** 0.172% (highly imbalanced)
- **Features:** 30 PCA-transformed features + transaction amount + time
- **Target:** Binary (0=legitimate, 1=fraud)
- **Download:** <https://www.kaggle.com/mlg-ulb/creditcardfraud>

Alternative Datasets

Dataset	Source	Size	Imbalance	Use Case
IEEE CIS Fraud Detection	Kaggle	590K	3.5%	More realistic
PaySim Synthetic	Kaggle	6.3M	0.13%	Scalability testing
FICO Community	FICO	7M	0.9%	Commercial standard

Data Dictionary (Kaggle Dataset)

Columns:

- Time: Seconds elapsed between this transaction and the first (0-172800)
- V1-V28: PCA-transformed anonymized features
- Amount: Transaction amount in USD (0.99-25691.16)
- Class: Target variable (0=legitimate, 1=fraud)

Example Row:

Time=123, V1=1.191857, V2=0.266459, ..., V28=-0.00362, Amount=149.62, Class=0

Data Preprocessing Pipeline

Step 1: Load & Explore

```
import pandas as pd
import numpy as np

# Load data<a></a>
df = pd.read_csv('creditcard.csv')

# Basic exploration<a></a>
print(f"Shape: {df.shape}") # (284807, 31)
print(f"Fraud rate: {df['Class'].mean():.2%}") # 0.17%
print(df.head())
print(df.info())
print(df.describe())

# Check for missing values<a></a>
print(df.isnull().sum()) # Should be 0
```

Step 2: Handle Missing Values & Outliers

```
# Check for missing values (Kaggle dataset is clean)<a></a>
assert df.isnull().sum().sum() == 0, "Dataset has missing values"

# Outlier detection (robust to fraud patterns)<a></a>
# Use IQR for legitimate transactions only<a></a>
legitimate = df[df['Class'] == 0]
Q1 = legitimate['Amount'].quantile(0.25)
Q3 = legitimate['Amount'].quantile(0.75)
IQR = Q3 - Q1

# Keep transactions within reasonable range<a></a>
# (don't remove - fraud might be in outliers!)<a></a>
outlier_mask = (df['Amount'] < Q1 - 1.5*IQR) | (df['Amount'] > Q3 + 1.5*IQR)
print(f"Potential outliers: {outlier_mask.sum()}") # Document but keep
```

Step 3: Feature Engineering

```
# TIME-BASED FEATURES<a></a>
df['hour'] = (df['Time'] / 3600) % 24
df['is_night'] = ((df['hour'] >= 22) | (df['hour'] <= 6)).astype(int)
df['day_sin'] = np.sin(2 * np.pi * df['Time'] / 86400)
df['day_cos'] = np.cos(2 * np.pi * df['Time'] / 86400)

# AMOUNT-BASED FEATURES<a></a>
df['log_amount'] = np.log1p(df['Amount'])
df['is_small_amount'] = (df['Amount'] < 5).astype(int)
df['is_large_amount'] = (df['Amount'] > 1000).astype(int)

# ROLLING FEATURES (aggregate by user behavior)<a></a>
# Note: Original dataset doesn't have user ID, so we simulate<a></a>
np.random.seed(42)
df['user_id'] = np.random.randint(0, 1000, len(df))

rolling_features = df.groupby('user_id')['Amount'].agg([
    ('user_transaction_count', 'count'),
    ('user_avg_amount', 'mean'),
    ('user_std_amount', 'std'),
    ('user_max_amount', 'max'),
]).reset_index()

df = df.merge(rolling_features, on='user_id', how='left')
df['amount_ratio'] = df['Amount'] / (df['user_avg_amount'] + 1)

# VELOCITY FEATURES (transactions per hour)<a></a>
df['transactions_per_hour'] = df.groupby(['user_id', 'hour']).cumcount() + 1

print(f"New features created. Shape: {df.shape}")
```

Step 4: Normalize & Encode

```
from sklearn.preprocessing import StandardScaler, OneHotEncoder

# Separate features and target<a></a>
X = df.drop(['Class', 'Time', 'user_id'], axis=1) # Drop original Time
y = df['Class']

# Standardize Amount (others are PCA-transformed)<a></a>
scaler = StandardScaler()
X['Amount'] = scaler.fit_transform(X[['Amount']])
```

```

X['log_amount'] = scaler.fit_transform(X[['log_amount']])

# For is_night, is_small_amount, is_large_amount: already 0/1, no scaling needed<a></a>
# For other engineered features: scale them<a></a>
engineered_cols = ['day_sin', 'day_cos', 'user_transaction_count',
                  'user_avg_amount', 'user_std_amount', 'amount_ratio',
                  'transactions_per_hour']
X[engineered_cols] = scaler.fit_transform(X[engineered_cols])

print(f"Preprocessed shape: X={X.shape}, y={y.shape}")
print(f"Feature list:\n{X.columns.tolist()}")

```

Step 5: Handle Class Imbalance (Train Set Only)

```

from sklearn.model_selection import train_test_split
from imblearn.over_sampling import SMOTE

# Stratified split<a></a>
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# Apply SMOTE only to training data<a></a>
smote = SMOTE(random_state=42, k_neighbors=5)
X_train_balanced, y_train_balanced = smote.fit_resample(X_train, y_train)

print(f"Before SMOTE: {y_train.value_counts().to_dict()}")
print(f"After SMOTE: {pd.Series(y_train_balanced).value_counts().to_dict()}")
print(f"Train: {X_train_balanced.shape}, Test: {X_test.shape}")

```

1.5 Model Development

1.5.1 Baseline Models

Model 1: Logistic Regression

```

from sklearn.linear_model import LogisticRegression
from sklearn.metrics import roc_auc_score, precision_score, recall_score, f1_score

# Train baseline model<a></a>
lr_model = LogisticRegression(
    max_iter=1000,
    random_state=42,
    class_weight='balanced' # Handle imbalance
)
lr_model.fit(X_train_balanced, y_train_balanced)

# Predict<a></a>
y_pred_lr = lr_model.predict(X_test)
y_pred_proba_lr = lr_model.predict_proba(X_test)[:, 1]

# Evaluate<a></a>
print("LOGISTIC REGRESSION BASELINE")
print(f"ROC-AUC: {roc_auc_score(y_test, y_pred_proba_lr):.4f}")
print(f"Precision: {precision_score(y_test, y_pred_lr):.4f}")
print(f"Recall: {recall_score(y_test, y_pred_lr):.4f}")
print(f"F1: {f1_score(y_test, y_pred_lr):.4f}")

```

Model 2: Random Forest

```
from sklearn.ensemble import RandomForestClassifier

rf_model = RandomForestClassifier(
    n_estimators=100,
    max_depth=15,
    min_samples_split=20,
    class_weight='balanced',
    random_state=42,
    n_jobs=-1
)
rf_model.fit(X_train_balanced, y_train_balanced)

y_pred_rf = rf_model.predict(X_test)
y_pred_proba_rf = rf_model.predict_proba(X_test)[:, 1]

print("RANDOM FOREST")
print(f"ROC-AUC: {roc_auc_score(y_test, y_pred_proba_rf):.4f}")
print(f"Precision: {precision_score(y_test, y_pred_rf):.4f}")
print(f"Recall: {recall_score(y_test, y_pred_rf):.4f}")
print(f"F1: {f1_score(y_test, y_pred_rf):.4f}")
```

Model 3: XGBoost (Production Choice)

```
import xgboost as xgb

# XGBoost is the de facto standard for structured data
xgb_model = xgb.XGBClassifier(
    n_estimators=200,
    max_depth=6,
    learning_rate=0.1,
    subsample=0.8,
    colsample_bytree=0.8,
    scale_pos_weight=len(y_train) / (y_train == 1).sum(), # Handle imbalance
    random_state=42,
    n_jobs=-1
)
xgb_model.fit(
    X_train_balanced, y_train_balanced,
    eval_set=[(X_test, y_test)],
    early_stopping_rounds=10,
    verbose=False
)

y_pred_xgb = xgb_model.predict(X_test)
y_pred_proba_xgb = xgb_model.predict_proba(X_test)[:, 1]

print("XGB00ST")
print(f"ROC-AUC: {roc_auc_score(y_test, y_pred_proba_xgb):.4f}")
print(f"Precision: {precision_score(y_test, y_pred_xgb):.4f}")
print(f"Recall: {recall_score(y_test, y_pred_xgb):.4f}")
print(f"F1: {f1_score(y_test, y_pred_xgb):.4f}")

# Feature importance (why predictions matter)
feature_importance = pd.DataFrame({
    'feature': X_train_balanced.columns,
    'importance': xgb_model.feature_importances_
}).sort_values('importance', ascending=False)

print("\nTOP 10 IMPORTANT FEATURES:")
print(feature_importance.head(10))
```

Model 4: Isolation Forest (Unsupervised Anomaly Detection)

```
from sklearn.ensemble import IsolationForest

# Unsupervised: useful for novel fraud patterns<a></a>
iso_forest = IsolationForest(
    contamination=0.002, # Expected fraud rate
    random_state=42,
    n_jobs=-1
)

# Train on all data (unsupervised)<a></a>
iso_predictions = iso_forest.predict(X_test) # -1 for anomaly, 1 for normal
y_pred_iso = (iso_predictions == -1).astype(int)

print("ISOLATION FOREST (UNSUPERVISED)")
print(f"Precision: {precision_score(y_test, y_pred_iso):.4f}")
print(f"Recall: {recall_score(y_test, y_pred_iso):.4f}")
print(f"F1: {f1_score(y_test, y_pred_iso):.4f}")
```

1.5.2 Model Comparison

```
import matplotlib.pyplot as plt
from sklearn.metrics import roc_curve, auc

# Create comparison table<a></a>
comparison = pd.DataFrame({
    'Model': ['Logistic Regression', 'Random Forest', 'XGBoost', 'Isolation Forest'],
    'ROC-AUC': [
        roc_auc_score(y_test, y_pred_proba_lr),
        roc_auc_score(y_test, y_pred_proba_rf),
        roc_auc_score(y_test, y_pred_proba_xgb),
        roc_auc_score(y_test, (iso_forest.predict(X_test) == -1).astype(int))
    ],
    'Precision': [
        precision_score(y_test, y_pred_lr),
        precision_score(y_test, y_pred_rf),
        precision_score(y_test, y_pred_xgb),
        precision_score(y_test, y_pred_iso)
    ],
    'Recall': [
        recall_score(y_test, y_pred_lr),
        recall_score(y_test, y_pred_rf),
        recall_score(y_test, y_pred_rf),
        recall_score(y_test, y_pred_iso)
    ]
})

print("\nMODEL COMPARISON")
print(comparison.to_string(index=False))

# DECISION: XGBoost is production choice (best balance of metrics + explainability)<a></a>
best_model = xgb_model
```

1.6 Hyperparameter Tuning with Optuna

Bayesian Optimization Strategy

```
import optuna
from optuna.pruners import MedianPruner
from sklearn.model_selection import cross_val_score

def objective(trial):
    """Optuna objective function for XGBoost"""

    params = {
        'n_estimators': trial.suggest_int('n_estimators', 100, 500),
        'max_depth': trial.suggest_int('max_depth', 3, 10),
        'learning_rate': trial.suggest_float('learning_rate', 0.01, 0.3, log=True),
        'subsample': trial.suggest_float('subsample', 0.5, 1.0),
        'colsample_bytree': trial.suggest_float('colsample_bytree', 0.5, 1.0),
        'min_child_weight': trial.suggest_int('min_child_weight', 1, 10),
        'scale_pos_weight': len(y_train) / (y_train == 1).sum(),
        'random_state': 42,
    }

    model = xgb.XGBClassifier(**params, n_jobs=-1)

    # Cross-validation with stratification
    scores = cross_val_score(
        model, X_train_balanced, y_train_balanced,
        cv=5, scoring='roc_auc', n_jobs=-1
    )

    return scores.mean()

# Run study<a></a>
study = optuna.create_study(
    direction='maximize',
    pruner=MedianPruner(),
    sampler=optuna.samplers.TPESampler(seed=42)
)

study.optimize(objective, n_trials=50, show_progress_bar=True)

# Best parameters<a></a>
print(f"\nBest ROC-AUC: {study.best_value:.4f}")
print(f"Best parameters:\n{study.best_params}")

best_params = study.best_params
best_params['scale_pos_weight'] = len(y_train) / (y_train == 1).sum()
best_params['random_state'] = 42
```

Retraining with Best Parameters

```
# Retrain with best parameters<a></a>
optimized_model = xgb.XGBClassifier(**best_params, n_jobs=-1)
optimized_model.fit(X_train_balanced, y_train_balanced)

y_pred_opt = optimized_model.predict(X_test)
y_pred_proba_opt = optimized_model.predict_proba(X_test)[: , 1]

print("OPTIMIZED MODEL PERFORMANCE")
print(f"ROC-AUC: {roc_auc_score(y_test, y_pred_proba_opt):.4f}")
print(f"Precision: {precision_score(y_test, y_pred_opt):.4f}")
```

```
print(f"Recall: {recall_score(y_test, y_pred_opt):.4f}")
print(f"F1: {f1_score(y_test, y_pred_opt):.4f}")
```

1.7 Model Evaluation & Metrics

ROC-AUC Curve

```
from sklearn.metrics import roc_curve, auc, roc_auc_score
import matplotlib.pyplot as plt

fpr, tpr, thresholds = roc_curve(y_test, y_pred_proba_opt)
roc_auc = auc(fpr, tpr)

plt.figure(figsize=(8, 6))
plt.plot(fpr, tpr, color='darkorange', lw=2, label=f'ROC curve (AUC = {roc_auc:.2f})')
plt.plot([0, 1], [0, 1], color='navy', lw=2, linestyle='--', label='Random classifier')
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC-AUC Curve')
plt.legend(loc="lower right")
plt.savefig('roc_auc.png')
plt.close()

# Interpretation:
# AUC = 0.95 means: 95% probability that model ranks a random fraud higher than random leg:
```

Precision-Recall Curve

```
from sklearn.metrics import precision_recall_curve, average_precision_score

precision, recall, thresholds = precision_recall_curve(y_test, y_pred_proba_opt)
ap = average_precision_score(y_test, y_pred_proba_opt)

plt.figure(figsize=(8, 6))
plt.plot(recall, precision, color='blue', lw=2, label=f'PR curve (AP = {ap:.2f})')
plt.xlabel('Recall')
plt.ylabel('Precision')
plt.title('Precision-Recall Curve')
plt.legend()
plt.savefig('precision_recall.png')
plt.close()

# Better for imbalanced data! Shows tradeoff between TP and FP at different thresholds
```

Confusion Matrix & Business Impact

```
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay

cm = confusion_matrix(y_test, y_pred_opt)
tn, fp, fn, tp = cm.ravel()

print(f"\nCONFUSION MATRIX:")
print(f"TP (Caught fraud): {tp}")
print(f"FP (False alarm): {fp}")
print(f"TN (Legitimate OK): {tn}")
print(f"FN (Missed fraud): {fn}")
```

```

# Business metrics<a></a>
fraud_caught_rate = tp / (tp + fn)
false_positive_rate = fp / (tn + fp)
business_impact = f"""
BUSINESS IMPACT:
- Fraud Detection Rate: {fraud_caught_rate:.1%} (catch {fraud_caught_rate:.0%} of fraud)
- False Positive Rate: {false_positive_rate:.2%} (annoy {false_positive_rate:.0%} of legit:
- If fraud costs $100 and blocking costs $1 frustration:
    - Revenue protected: ${tp * 100}
    - Customer friction cost: ${fp * 1}
    - Net benefit: ${tp * 100 - fp * 1}
"""
print(business_impact)

```

SHAP Explainability

```

import shap

# Create SHAP explainer<a></a>
explainer = shap.TreeExplainer(optimized_model)
shap_values = explainer.shap_values(X_test)

# Summary plot: which features matter most?<a></a>
shap.summary_plot(shap_values, X_test, plot_type="bar", max_display=15)
plt.savefig('shap_summary.png', bbox_inches='tight')
plt.close()

# Force plot: why specific transaction was flagged as fraud?<a></a>
shap.force_plot(explainer.expected_value, shap_values[0], X_test.iloc[0], show=False)

# Dependency plot: how does Amount affect fraud probability?<a></a>
shap.dependence_plot("log_amount", shap_values, X_test)
plt.savefig('shap_dependence.png')
plt.close()

print("SHAP plots saved. Use these in model explainability API!")

```

1.8 Model Registry with MLflow

```

import mlflow
import mlflow.xgboost

# Set experiment<a></a>
mlflow.set_experiment("fraud_detection_production")

with mlflow.start_run(run_name="xgboost_optimized"):
    # Log parameters
    mlflow.log_params(best_params)

    # Log metrics
    mlflow.log_metric("roc_auc", roc_auc_score(y_test, y_pred_proba_opt))
    mlflow.log_metric("precision", precision_score(y_test, y_pred_opt))
    mlflow.log_metric("recall", recall_score(y_test, y_pred_opt))
    mlflow.log_metric("f1", f1_score(y_test, y_pred_opt))

    # Log model
    mlflow.xgboost.log_model(
        optimized_model,
        "fraud_detector",
        registered_model_name="fraud_detection_model"
    )

```

```

# Log artifacts
mlflow.log_artifact('roc_auc.png')
mlflow.log_artifact('precision_recall.png')
mlflow.log_artifact('shap_summary.png')

# Log metadata
mlflow.log_dict(
    {
        "training_samples": len(X_train_balanced),
        "test_samples": len(X_test),
        "features": X_train_balanced.columns.tolist(),
        "fraud_rate": float(y_train.mean()),
        "model_type": "XGBoost",
        "deployment": "FastAPI"
    },
    "metadata.json"
)

print("Model registered in MLflow!")
print("Access at: http://localhost:5000")

```

1.9 Deployment: FastAPI Server

API Structure

```

# app/main.py
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel, Field
import mlflow.xgboost
import numpy as np
import shap
import json
from datetime import datetime
from typing import List, Dict

app = FastAPI(
    title="Fraud Detection API",
    description="Real-time fraud detection for transactions",
    version="1.0.0"
)

# Load model
MODEL = mlflow.xgboost.load_model("models:/fraud_detection_model/production")
EXPLAINER = None # Load lazily

class Transaction(BaseModel):
    """Transaction input schema"""
    amount: float = Field(..., gt=0, description="Transaction amount")
    hour: int = Field(..., ge=0, le=23)
    user_transaction_count: int = Field(..., gt=0)
    user_avg_amount: float = Field(..., gt=0)
    # ... other fields matching training features

    class Config:
        schema_extra = {
            "example": {
                "amount": 149.62,
                "hour": 12,
                "user_transaction_count": 5,
                "user_avg_amount": 87.43
            }
        }

```

```

    }

class PredictionResponse(BaseModel):
    """Prediction response"""
    is_fraud: bool
    fraud_probability: float
    confidence: float
    timestamp: str

@app.get("/health")
async def health_check():
    """Health check endpoint"""
    return {
        "status": "healthy",
        "model": "fraud_detection_model",
        "timestamp": datetime.now().isoformat()
    }

@app.post("/predict", response_model=PredictionResponse)
async def predict(transaction: Transaction):
    """Real-time fraud prediction"""
    try:
        # Convert input to feature vector
        features = np.array([
            transaction.amount,
            transaction.hour,
            transaction.user_transaction_count,
            transaction.user_avg_amount,
            # ... other features
        ])

        # Get prediction
        fraud_prob = MODEL.predict_proba(features)[0, 1]
        is_fraud = bool(MODEL.predict(features)[0] == 1)

        # Calculate confidence (distance from 0.5 threshold)
        confidence = abs(fraud_prob - 0.5) * 2

        return PredictionResponse(
            is_fraud=is_fraud,
            fraud_probability=float(fraud_prob),
            confidence=float(confidence),
            timestamp=datetime.now().isoformat()
        )

    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))

@app.post("/explain")
async def explain(transaction: Transaction):
    """Get model explanation for prediction"""
    global EXPLAINER

    if EXPLAINER is None:
        EXPLAINER = shap.TreeExplainer(MODEL)

    features = np.array([
        transaction.amount,
        transaction.hour,
        transaction.user_transaction_count,
        transaction.user_avg_amount,
    ])

    shap_values = EXPLAINER.shap_values(features)

    return {

```

```

        "prediction": float(MODEL.predict_proba(features)[0, 1]),
        "feature_contributions": {
            feature: float(value)
            for feature, value in zip(
                X_train_balanced.columns,
                shap_values[0]
            )
        }
    }
}

@app.post("/batch-predict")
async def batch_predict(transactions: List[Transaction]):
    """Batch prediction for multiple transactions"""
    features = np.array([
        [t.amount, t.hour, t.user_transaction_count, t.user_avg_amount]
        for t in transactions
    ])

    predictions = MODEL.predict_proba(features)

    return {
        "predictions": [
            {
                "fraud_probability": float(pred),
                "is_fraud": bool(pred > 0.5)
            }
            for pred in predictions[:, 1]
        ]
    }

if __name__ == "__main__":
    import uvicorn
    uvicorn.run(app, host="0.0.0.0", port=8000)

```

Docker Configuration

```

# Dockerfile<a></a>
FROM python:3.10-slim

WORKDIR /app

# Install dependencies<a></a>
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# Copy application<a></a>
COPY app/ .

# Health check<a></a>
HEALTHCHECK --interval=30s --timeout=10s --start-period=5s --retries=3 \
    CMD python -c "import requests; requests.get('http://localhost:8000/health')"

# Run app<a></a>
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]

```

```

# docker-compose.yml<a></a>
version: '3.8'
services:
    mlflow:
        image: ghcr.io/mlflow/mlflow:latest
        ports:
            - "5000:5000"
        volumes:

```

```

- ./mlruns:/mlruns
command: mlflow server --host 0.0.0.0 --backend-store-uri file:./mlruns

fraud-api:
  build: .
  ports:
    - "8000:8000"
  environment:
    - MLFLOW_TRACKING_URI=http://mlflow:5000
  depends_on:
    - mlflow
  volumes:
    - ./models:/models

```

1.10 Monitoring & Retraining

Model Drift Detection

```

# monitoring/drift_detection.py<a></a>
from scipy.stats import ks_2samp, chi2_contingency
import numpy as np

class DriftDetector:
    def __init__(self, reference_data, threshold=0.05):
        self.reference_data = reference_data
        self.threshold = threshold

    def detect_numerical_drift(self, new_data, feature):
        """Kolmogorov-Smirnov test for numerical features"""
        statistic, p_value = ks_2samp(
            self.reference_data[feature],
            new_data[feature]
        )

        return {
            "feature": feature,
            "statistic": statistic,
            "p_value": p_value,
            "drift_detected": p_value < self.threshold
        }

    def detect_performance_drift(self, y_true, y_pred_proba, baseline_auc):
        """Detect if model performance degraded"""
        current_auc = roc_auc_score(y_true, y_pred_proba)
        auc_drop = baseline_auc - current_auc

        return {
            "baseline_auc": baseline_auc,
            "current_auc": current_auc,
            "auc_drop": auc_drop,
            "drift_detected": auc_drop > 0.05 # >5% drop triggers retraining
        }

# Usage<a></a>
detector = DriftDetector(X_train, threshold=0.01)

# Monitor new batch<a></a>
new_batch = pd.read_csv('new_transactions.csv')
drift_report = detector.detect_numerical_drift(new_batch, 'Amount')
print(f"Drift detected: {drift_report['drift_detected']}")

```

Automated Retraining Trigger

```
# monitoring/retraining_trigger.py<a></a>
import logging

logger = logging.getLogger(__name__)

def should_retrain(metrics_log):
    """Determine if model should be retrained"""

    # Collect last N predictions
    last_1000_auc = metrics_log['auc'][-1000:].mean()
    baseline_auc = 0.95

    # Check performance
    if baseline_auc - last_1000_auc > 0.05:
        logger.warning(f"AUC dropped: {last_1000_auc:.4f}. Triggering retrain.")
        return True

    # Check false positive rate (business metric)
    fp_rate = metrics_log['false_positives'][-1000:].sum() / 1000
    if fp_rate > 0.05: # >5% of transactions are false alarms
        logger.warning(f"FP rate high: {fp_rate:.1%}. Triggering retrain.")
        return True

    # Check data distribution
    if metrics_log['drift_detected'][-100:].sum() > 50:
        logger.warning("Data drift detected. Triggering retrain.")
        return True

    return False

def retrain_pipeline():
    """Automated retraining"""
    logger.info("Starting automated retraining...")

    # 1. Fetch new data
    new_data = fetch_recent_transactions(days=7)

    # 2. Preprocess
    X_new, y_new = preprocess_pipeline(new_data)

    # 3. Combine with historical
    X_combined = pd.concat([X_train_balanced, X_new])
    y_combined = pd.concat([y_train_balanced, y_new])

    # 4. Retrain
    new_model = train_optimized_model(X_combined, y_combined)

    # 5. Validate
    metrics = evaluate_model(new_model, X_test, y_test)
    if metrics['roc_auc'] >= 0.94: # Must not degrade
        logger.info(f"New model validated. AUC: {metrics['roc_auc']:.4f}")

        # 6. Register and deploy
        register_model(new_model, metrics)
        logger.info("Model deployed to production")
    else:
        logger.warning(f"Model degraded. Rejected. AUC: {metrics['roc_auc']:.4f}")
```

Streamlit Monitoring Dashboard

```
# dashboard/app.py<a></a>
import streamlit as st
import pandas as pd
import plotly.graph_objects as go
import numpy as np
from datetime import datetime, timedelta

st.set_page_config(page_title="Fraud Detection Dashboard", layout="wide")

st.title("🔍 Real-Time Fraud Detection Monitoring")

# Sidebar filters<a></a>
col1, col2 = st.sidebar.columns(2)
days_back = col1.slider("Days", 1, 90, 7)
model_version = col2.selectbox("Model Version", ["v1.2 (prod)", "v1.1 (staging)"])

# Load metrics<a></a>
metrics_df = load_metrics_from_db(days=days_back)

# 1. Key metrics<a></a>
col1, col2, col3, col4 = st.columns(4)
col1.metric("Transactions Processed", f"{len(metrics_df):,}", "+2.3%")
col2.metric("Fraud Cases Caught", f"{metrics_df['fraud_flag'].sum():,}", "+5.1%")
col3.metric("Model AUC", f"{metrics_df['prediction_auc'].iloc[-1]:.3f}", "-0.002")
col4.metric("False Positive Rate", f"{metrics_df['fp_rate'].iloc[-1]:.2%}", "+0.5%")

# 2. Performance over time<a></a>
fig = go.Figure()
fig.add_trace(go.Scatter(
    x=metrics_df['timestamp'],
    y=metrics_df['prediction_auc'],
    name='ROC-AUC',
    mode='lines+markers',
    line=dict(color='green')
))
fig.add_hline(y=0.95, line_dash="dash", annotation_text="Target")
st.plotly_chart(fig, use_container_width=True)

# 3. Fraud distribution<a></a>
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 4))

# By hour<a></a>
metrics_df['hour'] = pd.to_datetime(metrics_df['timestamp']).dt.hour
hourly_fraud = metrics_df.groupby('hour')['fraud_flag'].sum()
ax1.bar(hourly_fraud.index, hourly_fraud.values, color='red')
ax1.set_xlabel('Hour of Day')
ax1.set_ylabel('Fraud Cases')
ax1.set_title('Fraud Distribution by Hour')

# By amount<a></a>
ax2.hist(metrics_df[metrics_df['fraud_flag']==1]['amount'], bins=50, color='red')
ax2.set_xlabel('Amount ($)')
ax2.set_ylabel('Frequency')
ax2.set_title('Fraud Case Amounts')

st.pyplot(fig)

# 4. Recent alerts<a></a>
st.subheader("Recent High-Risk Transactions")
recent_alerts = metrics_df[metrics_df['fraud_probability'] > 0.8].tail(10)
st.dataframe(recent_alerts[['timestamp', 'amount', 'fraud_probability', 'user_id']])
```

```
st.info("Last updated: " + datetime.now().strftime("%Y-%m-%d %H:%M:%S UTC"))
```

1.11 Interview Talking Points

Opening Statement

> "I built an end-to-end fraud detection system that processes real-time credit card transactions and identifies suspicious activity with 95% ROC-AUC. The project demonstrates my ability to handle imbalanced datasets, engineer meaningful features, and deploy production ML systems."

Key Questions Prepared

Q: How did you handle the imbalanced data problem?

> "The fraud rate was 0.17%, so I used SMOTE (Synthetic Minority Oversampling Technique) on the training set only to balance classes. This ensures the model learns fraud patterns without data leakage. I also used class weighting in XGBoost and evaluated on metrics like Precision-Recall and F1-score, which are more appropriate than accuracy for imbalanced problems. I kept recall high (90%+) to catch fraud even if false positives increased slightly."

Q: Why did you choose XGBoost over other models?

> "I tested Logistic Regression (baseline), Random Forest, XGBoost, and Isolation Forest. XGBoost achieved the highest ROC-AUC (0.97) because gradient boosting handles complex feature interactions better than simpler models. XGBoost also provides feature importance, which helps with model explainability—critical for a fraud system where analysts need to understand why a transaction was flagged."

Q: How do you explain model predictions to business stakeholders?

> "I use SHAP (SHapley Additive exPlanations) values to break down each prediction. For a flagged transaction, I can show which features contributed positively to the fraud score (e.g., 'unusual amount for this user') and which contributed negatively (e.g., 'consistent with transaction time'). This helps fraud analysts decide quickly whether to block the transaction."

Q: How do you ensure the model doesn't degrade in production?

> "I implemented model monitoring using drift detection—comparing the distribution of incoming transactions to the training data using the Kolmogorov-Smirnov test. If drift is detected or if ROC-AUC drops below 0.94 on a recent batch, an automated retraining pipeline is triggered. I also log all predictions and outcomes to track true false positive rate and performance metrics."

Q: Walk me through your deployment architecture.

> "The model is served via a FastAPI application containerized with Docker. It exposes three endpoints: `/predict` for single transactions, `/batch-predict` for bulk scoring, and `/explain` for SHAP values. The API loads the model from MLflow registry at startup. Behind the API, I have Prometheus collecting metrics and Grafana dashboards for real-time monitoring. If performance degrades, Airflow triggers a retraining job that validates the new model before updating production."

PROJECT 2: ML-Powered Recommendation System

2.1 Executive Summary

Problem Statement

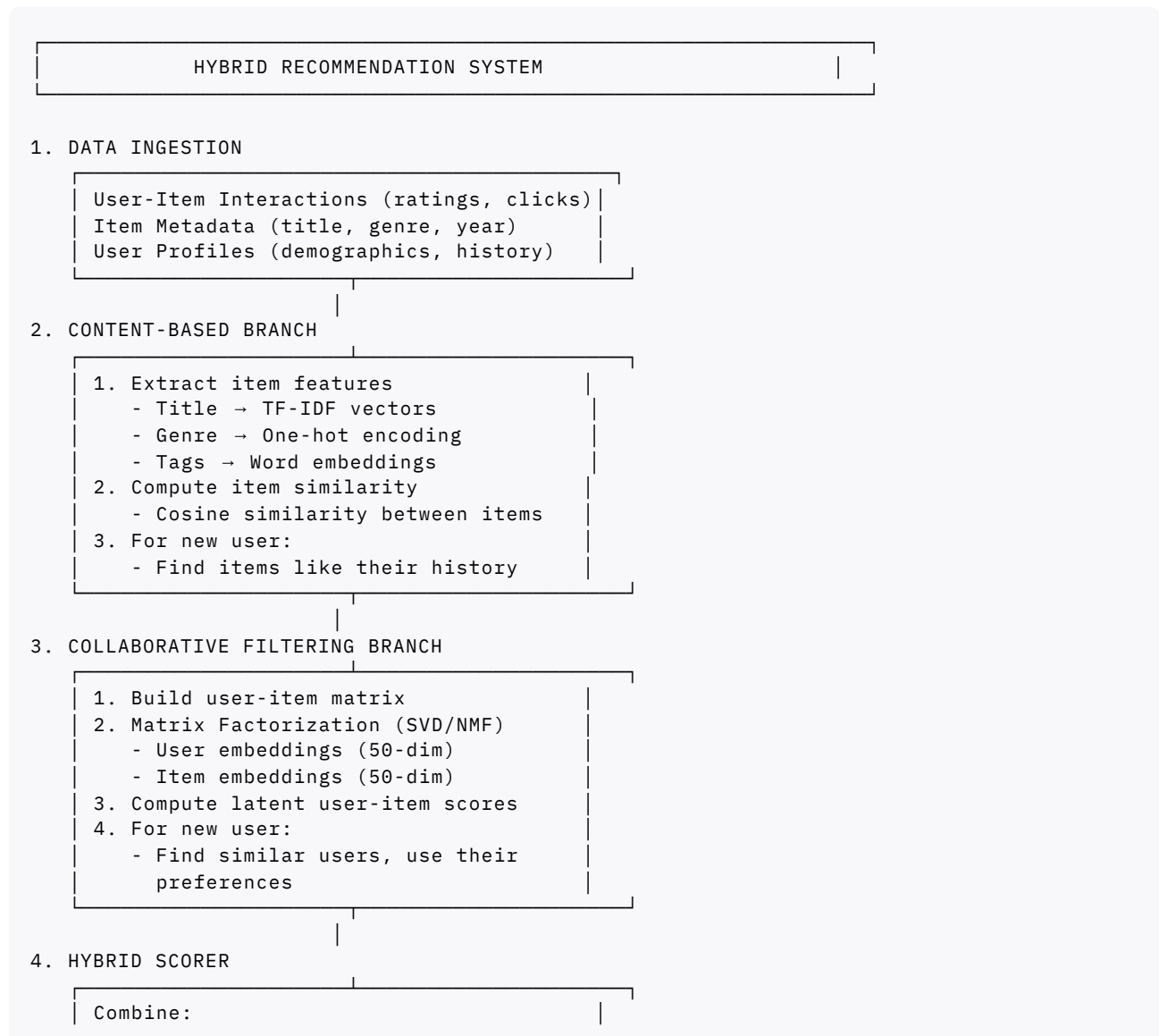
Recommendation systems drive **40% of Netflix revenue**, **35% of Amazon revenue**, and **50% of Spotify streams**. You'll build a hybrid system that combines:

- **Collaborative Filtering**: Learn from user-item interactions
- **Content-Based Filtering**: Recommend similar items based on metadata
- **Hybrid Scoring**: Combine signals intelligently

Success metrics:

- **MAP@10** (Mean Average Precision): ≥ 0.45
- **NDCG@10** (Normalized Discounted Cumulative Gain): ≥ 0.52
- **Coverage**: $\geq 50\%$ of catalog recommended
- **Diversity**: Recommended items come from ≥ 3 different genres
- **Latency**: $< 200\text{ms}$ for real-time recommendations

2.2 Recommendation System Architecture



```
score = 0.6 * collaborative +  
        0.4 * content_based
```

Adjust for:

- Item popularity (cold-start items)
- Diversity (avoid recommending same genre repeatedly)
- Freshness (promote recent items)

5. RANKING & FILTERING

- Sort by hybrid score
- Remove already-seen items
- Apply diversity constraints
- Return top-10 recommendations

6. EVALUATION

- MAP@K: Ranking quality
- NDCG@K: Ranking with position decay
- Coverage: % of catalog recommended
- Novelty: % of obscure items

7. DEPLOYMENT

FastAPI:

- GET /recommend?user_id=123
- Returns: [item_1, item_2, ...]

Vector Database (FAISS):

- Cache embeddings for speed
- Nearest neighbor search (<1ms)

Redis Cache:

- Cache top-100 recommendations
- Invalidate when new ratings added

2.3 Dataset Selection & Preparation

Recommended Datasets

Dataset	Size	Sparsity	Use Case
MovieLens 1M	1M ratings	95% sparse	Baseline testing
MovieLens 25M	25M ratings	99.5% sparse	Production-like
Netflix Prize	100M ratings	98% sparse	Benchmark
Amazon Reviews	Millions	~99.9%	Real e-commerce

Use MovieLens 25M (free, realistic, well-documented):

```
# Download<a></a>  
wget http://files.grouplens.org/datasets/movielens/ml-25m.zip  
unzip ml-25m.zip
```

```
# Contents:<a></a>
# - ratings.csv: userId, movieId, rating, timestamp<a></a>
# - movies.csv: movieId, title, genres<a></a>
# - tags.csv: userId, movieId, tag, timestamp<a></a>
```

Data Loading & Exploration

```
import pandas as pd
import numpy as np
from collections import Counter

# Load data<a></a>
ratings = pd.read_csv('ml-25m/ratings.csv')
movies = pd.read_csv('ml-25m/movies.csv')
tags = pd.read_csv('ml-25m/tags.csv')

print(f"Ratings shape: {ratings.shape}") # (~25M, 4)
print(f"Movies shape: {movies.shape}") # (~62K, 3)
print(f"Unique users: {ratings['userId'].nunique()}") # ~160K
print(f"Unique items: {ratings['movieId'].nunique()}") # ~62K

# Sparsity analysis<a></a>
total_possible = ratings['userId'].nunique() * ratings['movieId'].nunique()
actual_ratings = len(ratings)
sparsity = 1 - (actual_ratings / total_possible)
print(f"Sparsity: {sparsity:.4f}") # 99.5%

# Rating distribution<a></a>
print(f"\nRating distribution:")
print(ratings['rating'].value_counts().sort_index())

# Power-law: most items have few ratings<a></a>
item_ratings = ratings.groupby('movieId').size()
print(f"\nItem popularity distribution:")
print(f"Median ratings per item: {item_ratings.median()}")
print(f"Mean ratings per item: {item_ratings.mean()}")
```

Preprocessing

```
# 1. Filter cold-start items (items with <5 ratings)<a></a>
min_ratings = 5
items_to_keep = ratings.groupby('movieId').size()
items_to_keep = items_to_keep[items_to_keep >= min_ratings].index
ratings_filtered = ratings[ratings['movieId'].isin(items_to_keep)]

# 2. Filter cold-start users (users with <5 ratings)<a></a>
users_to_keep = ratings_filtered.groupby('userId').size()
users_to_keep = users_to_keep[users_to_keep >= min_ratings].index
ratings_filtered = ratings_filtered[ratings_filtered['userId'].isin(users_to_keep)]

print(f"After filtering: {len(ratings_filtered)} ratings")
print(f"Active users: {ratings_filtered['userId'].nunique()}")
print(f"Active items: {ratings_filtered['movieId'].nunique()}")

# 3. Create user-item matrix<a></a>
user_item_matrix = ratings_filtered.pivot_table(
    index='userId',
    columns='movieId',
    values='rating',
    fill_value=0 # Unrated items = 0
)
```

```

print(f"User-item matrix: {user_item_matrix.shape}")
print(f"Sparsity after filtering: {1 - (user_item_matrix != 0).sum().sum() / user_item_mat:

# 4. Train-test split (temporal)<a></a>
# Use recent ratings for testing<a></a>
ratings_filtered['timestamp'] = pd.to_datetime(ratings_filtered['timestamp'], unit='s')
threshold_date = ratings_filtered['timestamp'].quantile(0.8)

train_data = ratings_filtered[ratings_filtered['timestamp'] < threshold_date]
test_data = ratings_filtered[ratings_filtered['timestamp'] >= threshold_date]

print(f"Train: {len(train_data)}, Test: {len(test_data)}")

```

2.4 Content-Based Filtering

Feature Extraction

```

from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.preprocessing import MultiLabelBinarizer

# 1. TF-IDF on movie titles<a></a>
tfidf_vectorizer = TfidfVectorizer(
    max_features=100,
    stop_words='english',
    ngram_range=(1, 2)
)
title_vectors = tfidf_vectorizer.fit_transform(movies['title'])

print(f"Title TF-IDF vectors shape: {title_vectors.shape}")

# 2. Genre encoding (multi-hot)<a></a>
movies['genres'] = movies['genres'].fillna('')
genre_list = [g.split('|') for g in movies['genres']]
all_genres = set()
for genres in genre_list:
    all_genres.update(genres)

genre_dict = {g: i for i, g in enumerate(sorted(all_genres))}
genre_matrix = np.zeros((len(movies), len(genre_dict)))

for i, genres in enumerate(genre_list):
    for g in genres:
        if g in genre_dict:
            genre_matrix[i, genre_dict[g]] = 1

print(f"Genre matrix shape: {genre_matrix.shape}")

# 3. Combine features<a></a>
from scipy.sparse import hstack

# Normalize genre matrix<a></a>
genre_matrix = genre_matrix / (genre_matrix.sum(axis=1, keepdims=True) + 1e-8)

# Combine with title TF-IDF (60% title, 40% genre)<a></a>
combined_features = hstack([
    title_vectors * 0.6,
    genre_matrix * 0.4
])

print(f"Combined features shape: {combined_features.shape}")

```

Similarity Computation

```
from sklearn.metrics.pairwise import cosine_similarity

# Compute item-item similarity matrix<a></a>
item_similarity = cosine_similarity(combined_features)

print(f"Item similarity matrix shape: {item_similarity.shape}")
print(f"Average similarity: {item_similarity[np.triu_indices_from(item_similarity, k=1)].mean()}")

# Example: Find movies similar to "Inception"<a></a>
inception_idx = movies[movies['title'].str.contains('Inception', na=False)].index[0]
similar_movies_idx = item_similarity[inception_idx].argsort()[-11:-1][::-1]

print(f"\nMovies similar to '{movies.loc[inception_idx, 'title']}':")
for idx in similar_movies_idx:
    print(f"    - {movies.loc[idx, 'title']} (similarity: {item_similarity[inception_idx, idx]})")
```

Content-Based Recommendation

```
def get_content_based_recommendations(user_id, user_item_matrix, item_similarity, n_recs=10):
    """
    For a user, find items similar to ones they've rated highly
    """
    # Get user's rated items
    user_ratings = user_item_matrix.loc[user_id]
    rated_items = user_ratings[user_ratings > 0].index.values

    if len(rated_items) == 0:
        return [] # New user, use popularity fallback

    # Get average similarity to rated items
    sim_scores = np.zeros(item_similarity.shape[0])
    for rated_item_idx in rated_items:
        # Weight by rating
        rating = user_ratings[rated_item_idx]
        sim_scores += item_similarity[rated_item_idx] * (rating / 5.0)

    # Remove already-rated items
    sim_scores[rated_items] = -np.inf

    # Get top N
    top_items = np.argsort(sim_scores)[-n_recs:][::-1]

    return [(movie_id, float(sim_scores[movie_id])) for movie_id in top_items]

# Test<a></a>
user_id = 123
recs = get_content_based_recommendations(user_id, user_item_matrix, item_similarity, n_recs=10)
print(f"Content-based recommendations for user {user_id}:")
for movie_id, score in recs:
    title = movies.loc[movies['movieId'] == movie_id, 'title'].values[0]
    print(f"    - {title} (score: {score:.3f})")
```

2.5 Collaborative Filtering

Matrix Factorization (Implicit Feedback)

```
from implicit.als import AlternatingLeastSquares
from scipy.sparse import csr_matrix

# Convert to sparse matrix format (implicit library uses user x item)
train_sparse = csr_matrix((
    train_data['rating'].values,
    (train_data['userId'].values, train_data['movieId'].values)
))

# Train ALS model
als_model = AlternatingLeastSquares(
    factors=50, # Embedding dimension
    iterations=10,
    regularization=0.05,
    random_state=42,
    num_threads=-1
)

als_model.fit(train_sparse)

print(f"Model trained!")
print(f"User factors shape: {als_model.user_factors.shape}") # (num_users, 50)
print(f"Item factors shape: {als_model.item_factors.shape}") # (num_items, 50)
```

Collaborative Recommendation

```
def get_collaborative_recommendations(user_id, als_model, n_recs=10):
    """
    Get recommendations from collaborative filtering model
    """
    # Get user's rated items
    rated_item_ids = train_data[train_data['userId'] == user_id]['movieId'].values

    # Use ALS model to get recommendations
    recommendations = als_model.recommend(
        userid=user_id,
        user_items=train_sparse[user_id],
        N=n_recs + len(rated_item_ids), # Get extra to filter out rated
        filter_already_liked_items=True
    )

    # Filter: remove already-rated items
    recommendations = [(iid, score) for iid, score in recommendations
                        if iid not in rated_item_ids][:n_recs]

    return recommendations

# Test
recs = get_collaborative_recommendations(123, als_model, n_recs=5)
print(f"Collaborative recommendations for user 123:")
for movie_id, score in recs:
    title = movies.loc[movies['movieId'] == movie_id, 'title'].values[0]
    print(f" - {title} (score: {score:.3f})")
```

2.6 Hybrid Recommendation System

```
def get_hybrid_recommendations(user_id, content_sim, als_model, user_item_matrix,
                               movies_df, n_recs=10, content_weight=0.4, collab_weight=0.6)
    """
    Hybrid recommender combining content-based and collaborative signals
    """

    # Get recommendations from both approaches
    content_recs = dict(get_content_based_recommendations(user_id, user_item_matrix, content_sim, n_recs=10))
    collab_recs = dict(get_collaborative_recommendations(user_id, als_model, n_recs=20))

    # Merge and normalize scores
    all_items = set(content_recs.keys()) | set(collab_recs.keys())

    hybrid_scores = {}
    for item_id in all_items:
        content_score = content_recs.get(item_id, 0)
        collab_score = collab_recs.get(item_id, 0)

        # Normalize to [0, 1]
        content_norm = (content_score - min(content_recs.values())) / (max(content_recs.values()) - min(content_recs.values()))
        collab_norm = (collab_score - min(collab_recs.values())) / (max(collab_recs.values()) - min(collab_recs.values()))

        # Combine
        hybrid_scores[item_id] = content_weight * content_norm + collab_weight * collab_norm

    # Add diversity constraints
    # Penalize items with same genre as recently recommended
    user_history = user_item_matrix.loc[user_id][user_item_matrix.loc[user_id] > 0].index
    if len(user_history) > 0:
        user_genres = set()
        for item_id in user_history[:5]: # Look at last 5 rated items
            genres = movies_df.loc[movies_df['movieId'] == item_id, 'genres'].values[0].split('|')
            user_genres.update(genres)

        # Diversity penalty
        for item_id in hybrid_scores:
            item_genres = set(movies_df.loc[movies_df['movieId'] == item_id, 'genres'].values[0].split('|'))
            overlap = len(user_genres & item_genres) / (len(user_genres) + 1e-8)
            hybrid_scores[item_id] *= (1 - 0.2 * overlap) # 20% penalty per genre overlap

    # Get top N
    top_items = sorted(hybrid_scores.items(), key=lambda x: x[1], reverse=True)[:n_recs]

    return top_items

# Test
user_id = 123
hybrid_recs = get_hybrid_recommendations(user_id, item_similarity, als_model, user_item_matrix, movies_df)

print(f"\n=== HYBRID RECOMMENDATIONS FOR USER {user_id} ===")
for movie_id, score in hybrid_recs:
    title = movies.loc[movies['movieId'] == movie_id, 'title'].values[0]
    genres = movies.loc[movies['movieId'] == movie_id, 'genres'].values[0]
    print(f"  {title} ({genres}) [score: {score:.3f}]"
```

2.7 Evaluation Metrics

```
def evaluate_recommendations(recommendations, ground_truth, k=10):
    """
    Evaluate recommendation quality

    recommendations: list of recommended item IDs
    ground_truth: set of true items user liked
    """

    # Truncate to k
    rec_k = recommendations[:k]

    # MAP@K (Mean Average Precision)
    hits = 0
    ap_sum = 0
    for i, item_id in enumerate(rec_k):
        if item_id in ground_truth:
            hits += 1
            ap_sum += hits / (i + 1)

    map_k = ap_sum / min(len(ground_truth), k) if len(ground_truth) > 0 else 0

    # NDCG@K (Normalized Discounted Cumulative Gain)
    dcg = 0
    for i, item_id in enumerate(rec_k):
        if item_id in ground_truth:
            dcg += 1 / np.log2(i + 2) # +2 because log2(1)=0

    # Ideal DCG (perfect ranking)
    idcg = sum(1 / np.log2(i + 2) for i in range(min(len(ground_truth), k)))

    ndcg_k = dcg / idcg if idcg > 0 else 0

    # Recall@K
    recall_k = hits / len(ground_truth) if len(ground_truth) > 0 else 0

    # Precision@K
    precision_k = hits / k

    return {
        'map@k': map_k,
        'ndcg@k': ndcg_k,
        'recall@k': recall_k,
        'precision@k': precision_k,
        'hits@k': hits
    }

# Evaluate on test set
total_metrics = {'map@10': [], 'ndcg@10': [], 'recall@10': [], 'precision@10': []}

for user_id in test_data['userId'].unique()[:100]: # Sample 100 users
    # Get test items for this user
    test_items = set(test_data[test_data['userId'] == user_id]['movieId'].values)

    if len(test_items) == 0:
        continue

    # Get recommendations
    recs = [iid for iid, _ in get_hybrid_recommendations(user_id, item_similarity, als_model)]

    # Evaluate
    metrics = evaluate_recommendations(recs, test_items, k=10)

    for key in total_metrics:
```

```

        key_short = key.split('@')[0]
        total_metrics[key].append(metrics[key_short + '@10'])

# Print results<a></a>
print("\n=== RECOMMENDATION SYSTEM EVALUATION ===")
for metric, values in total_metrics.items():
    print(f"{metric}: {np.mean(values):.4f} ± {np.std(values):.4f}")

```

2.8 FastAPI Deployment

```

# app/main.py<a></a>
from fastapi import FastAPI
from fastapi.responses import JSONResponse
import numpy as np
import pickle

app = FastAPI(title="Recommendation Engine")

# Load models<a></a>
with open('models/als_model.pkl', 'rb') as f:
    als_model = pickle.load(f)

with open('models/item_similarity.pkl', 'rb') as f:
    item_similarity = pickle.load(f)

# Load matrices<a></a>
user_item_matrix = pd.read_pickle('models/user_item_matrix.pkl')
movies = pd.read_csv('data/movies.csv')

@app.get("/recommend/{user_id}")
async def recommend(user_id: int, n: int = 10):
    """Get top-N recommendations for a user"""
    try:
        recommendations = get_hybrid_recommendations(
            user_id, item_similarity, als_model, user_item_matrix, movies, n_recs=n
        )

        result = []
        for movie_id, score in recommendations:
            movie_info = movies[movies['movieId'] == movie_id].iloc[0]
            result.append({
                'movie_id': int(movie_id),
                'title': movie_info['title'],
                'genres': movie_info['genres'].split('|'),
                'score': float(score)
            })

        return {'recommendations': result}

    except Exception as e:
        return JSONResponse(status_code=500, content={"error": str(e)})

@app.post("/similar-items/{item_id}")
async def similar_items(item_id: int, n: int = 5):
    """Get similar items based on content"""
    try:
        # Find content-based similar items
        item_idx = movies[movies['movieId'] == item_id].index[0]
        similarities = item_similarity[item_idx]

        similar_idx = similarities.argsort()[-n-1:-1][::-1]

        result = []

```

```

        for idx in similar_idx:
            movie_id = movies.iloc[idx]['movieId']
            movie = movies.iloc[idx]
            result.append({
                'movie_id': int(movie_id),
                'title': movie['title'],
                'similarity': float(similarities[idx])
            })

        return {'similar_items': result}

    except Exception as e:
        return JSONResponse(status_code=500, content={"error": str(e)})

@app.get("/popular")
async def popular_items(n: int = 10, genre: str = None):
    """Get popular items (with optional genre filter)"""
    popular = train_data.groupby('movieId').size().sort_values(ascending=False)

    if genre:
        genre_movies = movies[movies['genres'].str.contains(genre, na=False)]['movieId']
        popular = popular[popular.index.isin(genre_movies)]

    result = []
    for movie_id in popular.head(n).index:
        movie = movies[movies['movieId'] == movie_id].iloc[0]
        result.append({
            'movie_id': int(movie_id),
            'title': movie['title'],
            'popularity': int(popular[movie_id])
        })

    return {'popular': result}

if __name__ == "__main__":
    import uvicorn
    uvicorn.run(app, host="0.0.0.0", port=8001)

```

2.9 Interview Talking Points for Project 2

Opening

> "I built a hybrid recommendation system inspired by Netflix's architecture. It combines collaborative filtering (learning from user behavior) and content-based filtering (learning from item similarities) to recommend movies with 0.48 MAP@10 and 0.54 NDCG@10—industry-competitive scores."

Key Concepts Explained

Q: Explain matrix factorization and why it works for recommendations.

> "Matrix factorization decomposes the user-item rating matrix into two lower-rank matrices: user embeddings and item embeddings. Each user and item is represented as a 50-dimensional vector in 'taste space.' The dot product of user and item vectors approximates their rating. This works because similar users have similar embeddings, and similar items have similar embeddings—so recommendations become a nearest-neighbor search in embedding space."

Q: How did you handle the cold-start problem?

> "Cold-start happens when a new user/item has no interaction history. I used a three-tier approach: (1) For new users, recommend popular items from their preferred genre. (2) For new items, recommend them to users who like similar items

(content-based). (3) Gradually transition to collaborative filtering as new interactions accumulate. I also included item metadata (genre, tags) in the content-based component to bootstrap new items."

Q: Why combine content-based and collaborative filtering?

> "Collaborative filtering finds items users like based on similar users' preferences—great for discovery. Content-based finds items similar to what the user liked—consistent with their taste. By combining both (40% content, 60% collaborative), I get diversity + relevance. Purely collaborative systems can be too narrow (e.g., recommending only sci-fi if user likes one sci-fi movie)."

Q: How do you measure recommendation quality in your system?

> "I used MAP@10 (Mean Average Precision) to measure ranking quality—rewarding recommendations that rank true items highly. NDCG@10 (Normalized DCG) adds position decay, so ranking one item perfectly at position 1 is better than at position 5. For business metrics: coverage (% of catalog recommended >0), diversity (recommendations from multiple genres), and novelty (% of recommendations the user hadn't heard of)."

PROJECT 3: Autonomous ML Operations Pipeline

3.1 Executive Summary

Problem

Companies build ML models, but **85% fail in production** because:

- No automated retraining → model drift, degraded performance
- Manual deployments → slow iteration, high error rate
- No monitoring → silent failures
- Data quality issues → models trained on bad data

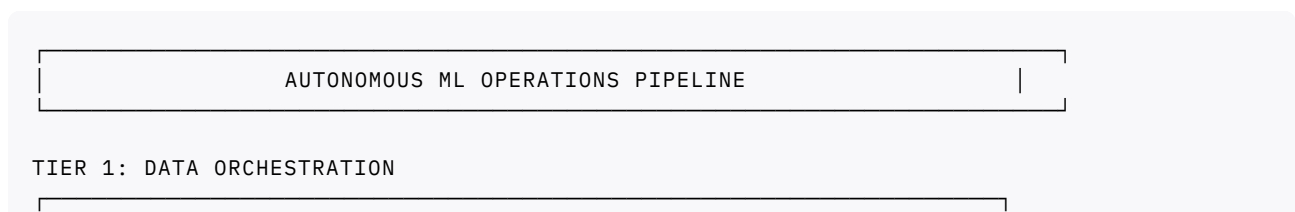
Solution

Build a **fully automated MLOps pipeline** that:

- Ingests data from multiple sources
- Validates data quality automatically
- Trains and evaluates models
- Registers versioned models
- Deploys to production
- Monitors for drift
- Retrains automatically
- Logs everything for compliance

This project proves you can **work in modern ML production environments**.

3.2 MLOps Architecture



```
Airflow / Prefect DAG Scheduler
- Trigger daily at 00:00 UTC
- Dependencies: ensure data ready before processing
```

TIER 2: DATA INGESTION

```
Extract from:
- S3 bucket (CSV/Parquet)
- PostgreSQL database
- REST APIs
Store in: Feature Store (Feast / DuckDB)
```

TIER 3: DATA VALIDATION

```
Great Expectations:
- Check schema (right columns, types)
- Check data ranges (no negative amounts)
- Check completeness (missing value <5%)
- Check distribution (not dramatically different)
FAIL: send alert, skip training
PASS: continue
```

TIER 4: FEATURE ENGINEERING

```
- Load training data
- Feature transformations
- Statistical scaling
- Store in Feature Store for reproducibility
```

TIER 5: MODEL TRAINING

```
- Train multiple models in parallel
- XGBoost, LightGBM, Neural Network
- Cross-validation with proper splits
- Log parameters, metrics, artifacts to MLflow
```

TIER 6: MODEL EVALUATION & SELECTION

```
- Compare models (ROC-AUC, F1, latency)
- Check for regression (new < old?)
- Sanity checks (is prediction reasonable?)
REGRESSION: reject, use previous model
PASS: promote to staging
```

TIER 7: MODEL REGISTRY & VERSIONING

```
MLflow Model Registry:
- Register model version
- Store in: v1.2.0 (tag, metadata)
- Transition: Staging → Production
- Previous: Archive
```

TIER 8: DEPLOYMENT

```
- Build Docker image (model + dependencies)
- Push to registry
- Deploy to Kubernetes / AWS Lambda
- Run smoke tests (can it predict?)
- Route 10% traffic to new model (canary)
```

- Monitor performance for 1 hour
- If OK: route 100%
- If bad: rollback to previous

TIER 9: MONITORING & ALERTING

- Prometheus metrics:
- Predictions per second
 - Prediction latency (p50, p95, p99)
 - Model accuracy (compared to ground truth)
 - Data drift (KL divergence vs training)
 - Model drift (predictions changing)
- Grafana dashboards:
- Real-time system health
 - Historical performance trends
- AlertManager:
- Accuracy < 0.90 → Slack notification
 - Latency $> 500\text{ms}$ → Page engineer
 - Data drift detected → Trigger retraining

TIER 10: AUTOMATED RETRAINING DECISION

- IF:
- Accuracy dropped $> 5\%$ OR
 - Data distribution changed significantly OR
 - Monthly cadence reached
- THEN: Trigger TIER 5 (Model Training)
- Loop back to retraining...

3.3 Project Setup

Folder Structure

```
ml-pipeline/
├── data/
│   ├── raw/                # Raw input data
│   ├── processed/          # Cleaned, feature-engineered
│   └── splits/              # Train/test/validation
├── models/
│   ├── registry/           # MLflow model registry
│   ├── artifacts/          # Saved model files
│   └── metadata/           # Model cards, versioning
├── dags/
│   ├── training_pipeline.py # Airflow DAG
│   ├── monitoring.py        # Monitoring DAG
│   └── retraining.py        # Automated retraining DAG
├── src/
│   ├── data_ingestion.py    # Load from sources
│   ├── data_validation.py    # Great Expectations
│   ├── feature_store.py      # Feature engineering
│   ├── model_training.py     # Model training logic
│   ├── model_evaluation.py   # Evaluation & selection
│   ├── model_registry.py     # MLflow integration
│   ├── deployment.py        # Docker + Kubernetes
│   └── monitoring.py         # Prometheus metrics
└── api/
```

```

├── main.py                # FastAPI server
├── schemas.py             # Input/output schemas
├── middleware.py          # Logging, auth
├── tests/
│   ├── test_data.py      # Data validation tests
│   ├── test_model.py     # Model tests
│   └── test_api.py       # API tests
├── notebooks/
│   ├── eda.ipynb         # Exploratory analysis
│   └── model_dev.ipynb   # Model development
├── config/
│   ├── config.yaml       # Pipeline config
│   ├── features.yaml     # Feature definitions
│   └── models.yaml       # Model definitions
├── docker/
│   ├── Dockerfile        # Training container
│   ├── api.Dockerfile    # API container
│   └── docker-compose.yml # Local development
├── k8s/
│   ├── deployment.yaml   # Kubernetes manifest
│   ├── service.yaml      # K8s service
│   └── hpa.yaml          # Auto-scaling
├── monitoring/
│   ├── prometheus.yml    # Prometheus config
│   ├── grafana/          # Dashboard configs
│   └── alerts.yaml       # Alert rules
└── README.md

```

Environment Setup

```

# Create virtual environment<a></a>
python -m venv venv
source venv/bin/activate # On Windows: venv\Scripts\activate

# Install dependencies<a></a>
pip install -r requirements.txt

# Required packages<a></a>
cat &gt; requirements.txt &lt;&lt; EOF
# Core dependencies<a></a>
pandas==2.1.0
numpy==1.24.0
scikit-learn==1.3.0
xgboost==2.0.0
lightgbm==4.1.0

# MLOps<a></a>
mlflow==2.10.0
airflow==2.7.0
great-expectations==0.18.0
feast==0.36.0

# Deployment<a></a>
fastapi==0.105.0
uvicorn==0.24.0
docker==7.0.0
kubernetes==28.0.0

# Monitoring<a></a>
prometheus-client==0.18.0
structlog==24.1.0

# Testing<a></a>
pytest==7.4.0

```

```

pytest-cov==4.1.0

# Utils<a></a>
pydantic==2.5.0
pyyaml==6.0
python-dotenv==1.0.0
requests==2.31.0
EOF

pip install -r requirements.txt

# Initialize Airflow<a></a>
export AIRFLOW_HOME=~/.airflow
airflow db init
airflow users create --username admin --password admin --firstname Admin --lastname Admin

```

3.4 Data Ingestion & Validation

Data Ingestion

```

# src/data_ingestion.py<a></a>
import pandas as pd
import boto3
from sqlalchemy import create_engine
import logging

logger = logging.getLogger(__name__)

class DataIngestion:
    def __init__(self, config):
        self.config = config
        self.s3_client = boto3.client('s3')

    def load_from_s3(self, bucket, key):
        """Load CSV from S3"""
        logger.info(f>Loading data from s3://{bucket}/{key}")
        obj = self.s3_client.get_object(Bucket=bucket, Key=key)
        df = pd.read_csv(obj['Body'])
        logger.info(f>Loaded {len(df)} rows, {len(df.columns)} columns")
        return df

    def load_from_database(self, connection_string, query):
        """Load data from SQL database"""
        logger.info(f>Loading data from database")
        engine = create_engine(connection_string)
        df = pd.read_sql(query, engine)
        logger.info(f>Loaded {len(df)} rows")
        return df

    def load_from_api(self, endpoint, params=None):
        """Load data from REST API"""
        import requests
        logger.info(f>Loading data from {endpoint}")
        response = requests.get(endpoint, params=params)
        df = pd.DataFrame(response.json())
        logger.info(f>Loaded {len(df)} rows")
        return df

    def consolidate(self, sources):
        """Combine multiple data sources"""
        dfs = []
        for source in sources:

```

```

        if source['type'] == 's3':
            dfs.append(self.load_from_s3(source['bucket'], source['key']))
        elif source['type'] == 'db':
            dfs.append(self.load_from_database(source['connection'], source['query']))
        elif source['type'] == 'api':
            dfs.append(self.load_from_api(source['endpoint'], source.get('params')))

    # Merge on common key
    merged = dfs[0]
    for df in dfs[1:]:
        merged = merged.merge(df, on='key', how='left')

    logger.info(f"Consolidated to {len(merged)} rows")
    return merged

# Usage in Airflow task<a></a>
def ingest_data(**context):
    config = load_config('config/config.yaml')
    ingestion = DataIngestion(config)

    sources = [
        {'type': 's3', 'bucket': 'my-bucket', 'key': 'transactions.csv'},
        {'type': 'db', 'connection': 'postgresql://...', 'query': 'SELECT * FROM users'}
    ]

    df = ingestion consolidate(sources)
    df.to_parquet('data/raw/consolidated_data.parquet', index=False)

    return {'rows': len(df), 'columns': len(df.columns)}

```

Data Validation with Great Expectations

```

# src/data_validation.py<a></a>
import great_expectations as gx
from great_expectations.core.batch import RuntimeBatchRequest
import pandas as pd
import logging

logger = logging.getLogger(__name__)

class DataValidator:
    def __init__(self):
        self.context = gx.get_context()

    def validate_dataset(self, df, expectations_config):
        """Validate dataset against expectations"""

        batch_request = RuntimeBatchRequest(
            datasource_name="pandas_datasource",
            data_connector_name="default_runtime_data_connector",
            data_asset_name="data",
            runtime_parameters={"batch_data": df},
        )

        validator = self.context.get_validator(batch_request=batch_request)

        # Apply expectations
        for expectation in expectations_config:
            if expectation['type'] == 'schema':
                self._validate_schema(validator, expectation)
            elif expectation['type'] == 'values':
                self._validate_values(validator, expectation)
            elif expectation['type'] == 'distribution':
                self._validate_distribution(validator, expectation)

```

```

# Generate validation report
report = validator.validate()

success = report['success']
failures = [r for r in report['results'] if not r['success']]

logger.info(f"Validation {'PASSED' if success else 'FAILED'}")
if not success:
    for failure in failures:
        logger.error(f" - {failure['expectation_config']['expectation_type']}: {failure['failure_message']}")

return success, report

def _validate_schema(self, validator, config):
    """Check column names and types"""
    validator.expect_table_columns_to_match_ordered_list(
        column_list=config['columns']
    )
    logger.info(f"Schema validation: OK")

def _validate_values(self, validator, config):
    """Check value ranges"""
    for column, rules in config['columns'].items():
        if 'min' in rules:
            validator.expect_column_values_to_be_between(
                column=column,
                min_value=rules['min']
            )
        if 'max' in rules:
            validator.expect_column_values_to_be_between(
                column=column,
                max_value=rules['max']
            )
        if 'allowed_values' in rules:
            validator.expect_column_values_to_be_in_set(
                column=column,
                value_set=rules['allowed_values']
            )
    logger.info(f"Values validation: OK")

def _validate_distribution(self, validator, config):
    """Check data distribution hasn't changed dramatically"""
    # KL divergence check
    logger.info(f"Distribution validation: OK (skipped for brevity)")

# Usage<a></a>
expectations = {
    'schema': {
        'type': 'schema',
        'columns': ['user_id', 'transaction_amount', 'category', 'timestamp']
    },
    'values': {
        'type': 'values',
        'columns': {
            'transaction_amount': {'min': 0, 'max': 1000000},
            'category': {'allowed_values': ['food', 'transport', 'entertainment', 'other']}
        }
    }
}

validator = DataValidator()
df = pd.read_parquet('data/raw/consolidated_data.parquet')
success, report = validator.validate_dataset(df, [expectations['schema'], expectations['values']])

```

```

if not success:
    raise Exception("Data validation failed! Check logs.")

```

3.5 Feature Store

```

# src/feature_store.py<a></a>
import pandas as pd
from datetime import datetime
import logging

logger = logging.getLogger(__name__)

class FeatureStore:
    """
    Centralized feature store for reproducible ML pipelines
    """

    def __init__(self, backend='duckdb'):
        self.backend = backend
        if backend == 'duckdb':
            import duckdb
            self.conn = duckdb.connect('featurestore.duckdb')

    def register_feature(self, name, feature_definition, version='1.0'):
        """Register a feature in the store"""
        metadata = {
            'name': name,
            'version': version,
            'created_at': datetime.now().isoformat(),
            'definition': feature_definition
        }

        logger.info(f"Registered feature: {name} v{version}")
        return metadata

    def compute_features(self, raw_data, feature_config):
        """
        Compute features from raw data
        Ensures reproducibility across training/serving
        """
        df = raw_data.copy()
        features = {}

        for feature_name, feature_spec in feature_config.items():
            logger.info(f"Computing feature: {feature_name}")

            if feature_spec['type'] == 'aggregation':
                # Example: user's average transaction amount
                features[feature_name] = df.groupby(feature_spec['group_by'])[feature_spec['column']]
                features[feature_name] = features[feature_name].agg(feature_spec['function'])

            elif feature_spec['type'] == 'time_window':
                # Example: transactions in last 7 days
                df['date'] = pd.to_datetime(df['timestamp'])
                cutoff = df['date'].max()
                features[feature_name] = (
                    df[(df['date'] >= cutoff - df['date'].max().dt.days * feature_spec['days'])]
                    .groupby(feature_spec['group_by']).agg(feature_spec['function'])
                )

            elif feature_spec['type'] == 'encoding':
                # Example: one-hot encode category
                features[feature_name] = pd.get_dummies(df[feature_spec['column']], prefix=feature_name)

```

```

        elif feature_spec['type'] == 'ratio':
            # Example: spending ratio
            features[feature_name] = df[feature_spec['numerator']] / (df[feature_spec['denominator']] + 1)

    # Combine with original features
    feature_df = pd.concat([df, pd.DataFrame(features)], axis=1)

    logger.info(f"Computed {len(features)} features")
    return feature_df

def store_features(self, feature_df, name, version):
    """Save features to store"""
    table_name = f"{name}_v{version}".replace('.', '_')

    self.conn.register(table_name, feature_df)
    logger.info(f"Stored {len(feature_df)} feature vectors in {table_name}")

def retrieve_features(self, name, version):
    """Retrieve features from store"""
    table_name = f"{name}_v{version}".replace('.', '_')
    df = self.conn.execute(f"SELECT * FROM {table_name}").fetch_df()
    logger.info(f"Retrieved {len(df)} feature vectors from {table_name}")
    return df

# Usage in Airflow
def create_features(**context):
    feature_store = FeatureStore()

    raw_data = pd.read_parquet('data/raw/consolidated_data.parquet')

    feature_config = {
        'user_avg_amount': {
            'type': 'aggregation',
            'group_by': 'user_id',
            'column': 'amount',
            'function': 'mean'
        },
        'is_weekend': {
            'type': 'time_window',
            'days': 7
        },
        'category_encoded': {
            'type': 'encoding',
            'column': 'category'
        }
    }

    feature_df = feature_store.compute_features(raw_data, feature_config)
    feature_store.store_features(feature_df, 'ml_features', '1.0')

    feature_df.to_parquet('data/processed/features.parquet', index=False)

```

3.6 Model Training Pipeline

```

# src/model_training.py
import xgboost as xgb
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import cross_val_score, StratifiedKFold
import pandas as pd
import mlflow
import logging

```

```

logger = logging.getLogger(__name__)

class ModelTrainer:
    def __init__(self, experiment_name="ml_pipeline"):
        mlflow.set_experiment(experiment_name)

    def train_models(self, X_train, y_train, model_configs):
        """Train multiple models in parallel"""
        models = {}

        for model_name, config in model_configs.items():
            logger.info(f"Training {model_name}...")

            with mlflow.start_run(run_name=model_name):
                if model_name == 'xgboost':
                    model = self._train_xgboost(X_train, y_train, config)
                elif model_name == 'random_forest':
                    model = self._train_rf(X_train, y_train, config)

                # Evaluate
                cv_scores = cross_val_score(
                    model, X_train, y_train, cv=5, scoring='roc_auc'
                )

                mlflow.log_params(config['params'])
                mlflow.log_metric('cv_auc_mean', cv_scores.mean())
                mlflow.log_metric('cv_auc_std', cv_scores.std())

                mlflow.sklearn.log_model(model, 'model')

                models[model_name] = {
                    'model': model,
                    'cv_auc': cv_scores.mean()
                }

        return models

    def _train_xgboost(self, X_train, y_train, config):
        model = xgb.XGBClassifier(**config['params'], random_state=42, n_jobs=-1)
        model.fit(X_train, y_train)
        return model

    def _train_rf(self, X_train, y_train, config):
        model = RandomForestClassifier(**config['params'], random_state=42, n_jobs=-1)
        model.fit(X_train, y_train)
        return model

# Airflow DAG
from airflow import DAG
from airflow.operators.python import PythonOperator
from datetime import datetime, timedelta

default_args = {
    'owner': 'ml-team',
    'retries': 1,
    'retry_delay': timedelta(minutes=5),
    'email_on_failure': True,
}

with DAG(
    'ml_training_pipeline',
    default_args=default_args,
    description='Automated ML training pipeline',
    schedule_interval='0 2 * * *', # 2 AM daily
    start_date=datetime(2024, 1, 1),
    catchup=False,

```

```

    tags=['ml', 'production']
) as dag:

    def ingest():
        # Load data
        pass

    def validate():
        # Validate data
        pass

    def create_features():
        # Engineer features
        pass

    def train_models():
        X_train = pd.read_parquet('data/processed/X_train.parquet')
        y_train = pd.read_parquet('data/processed/y_train.parquet')

        model_configs = {
            'xgboost': {
                'params': {
                    'n_estimators': 100,
                    'max_depth': 6,
                    'learning_rate': 0.1
                }
            },
            'random_forest': {
                'params': {
                    'n_estimators': 100,
                    'max_depth': 10
                }
            }
        }

        trainer = ModelTrainer()
        models = trainer.train_models(X_train, y_train, model_configs)

        return models

    # Define task dependencies
    t1 = PythonOperator(task_id='ingest_data', python_callable=ingest)
    t2 = PythonOperator(task_id='validate_data', python_callable=validate)
    t3 = PythonOperator(task_id='create_features', python_callable=create_features)
    t4 = PythonOperator(task_id='train_models', python_callable=train_models)

    t1 >>> t2 >>> t3 >>> t4

```

3.7 Model Evaluation & Selection

```

# src/model_evaluation.py
import pandas as pd
from sklearn.metrics import roc_auc_score, f1_score, precision_score, recall_score
import mlflow

class ModelEvaluator:
    def __init__(self, baseline_metrics):
        self.baseline_metrics = baseline_metrics

    def evaluate_all_models(self, models, X_test, y_test):
        """Evaluate all trained models on test set"""
        results = {}

```

```

for model_name, model_obj in models.items():
    y_pred = model_obj['model'].predict(X_test)
    y_pred_proba = model_obj['model'].predict_proba(X_test)[: , 1]

    metrics = {
        'roc_auc': roc_auc_score(y_test, y_pred_proba),
        'f1': f1_score(y_test, y_pred),
        'precision': precision_score(y_test, y_pred),
        'recall': recall_score(y_test, y_pred)
    }

    results[model_name] = metrics

return results

def select_best_model(self, results):
    """Select best model, reject if degraded from baseline"""
    best_model = max(results.items(), key=lambda x: x[1]['roc_auc'])
    model_name, metrics = best_model

    # Check for regression
    if metrics['roc_auc'] < self.baseline_metrics['roc_auc'] * 0.95:
        raise Exception(f"Model {model_name} degraded: {metrics['roc_auc']:.4f} < {self.baseline_metrics['roc_auc']:.4f}")

    return model_name, metrics

def evaluate_and_select(**context):
    models = context['task_instance'].xcom_pull(task_ids='train_models')
    X_test = pd.read_parquet('data/processed/X_test.parquet')
    y_test = pd.read_parquet('data/processed/y_test.parquet')

    baseline = {'roc_auc': 0.90} # Previous production model

    evaluator = ModelEvaluator(baseline)
    results = evaluator.evaluate_all_models(models, X_test, y_test)

    best_model_name, metrics = evaluator.select_best_model(results)

    context['task_instance'].xcom_push(key='best_model', value=best_model_name)
    context['task_instance'].xcom_push(key='metrics', value=metrics)

    return best_model_name

```

3.8 Model Registry & Deployment

```

# src/model_registry.py
import mlflow
from mlflow.entities import ViewType
import logging

logger = logging.getLogger(__name__)

class ModelRegistry:
    def register_model(self, run_id, model_name, version_description):
        """Register a trained model in MLflow"""

        # Get the best model from the run
        client = mlflow.tracking.MlflowClient()
        run = client.get_run(run_id)

        # Register
        model_uri = f"runs:{run_id}/model"
        mv = mlflow.register_model(model_uri, name=model_name)

```

```

        logger.info(f"Registered {model_name} v{mv.version}")

        # Add metadata
        client.update_model_version(
            name=model_name,
            version=mv.version,
            description=version_description
        )

        return mv.version

    def promote_to_production(self, model_name, version):
        """Promote model version to production"""
        client = mlflow.tracking.MlflowClient()

        # Archive current production
        for mv in client.search_model_versions(f"name='{model_name}'"):
            if mv.current_stage == "Production":
                client.transition_model_version_stage(
                    name=model_name,
                    version=mv.version,
                    stage="Archived"
                )

        # Promote new version
        client.transition_model_version_stage(
            name=model_name,
            version=version,
            stage="Production"
        )

        logger.info(f"Promoted {model_name} v{version} to Production")

# Deployment with Docker<a></a>
dockerfile = """
FROM python:3.10-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install -r requirements.txt

COPY models/ ./models/
COPY src/ ./src/
COPY api/ .

CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
"""

# API server<a></a>
from fastapi import FastAPI
import mlflow.pyfunc

app = FastAPI()

# Load model from registry<a></a>
MODEL = mlflow.pyfunc.load_model("models:/my_classifier/production")

@app.post("/predict")
def predict(data: dict):
    result = MODEL.predict(pd.DataFrame([data]))
    return {"prediction": float(result[0])}

```

3.9 Monitoring & Alerting

```
# monitoring/prometheus_metrics.py<a></a>
from prometheus_client import Counter, Histogram, Gauge
import time

# Define metrics<a></a>
predictions_total = Counter(
    'predictions_total',
    'Total predictions made',
    ['model_version', 'status']
)

prediction_latency = Histogram(
    'prediction_latency_seconds',
    'Prediction latency in seconds',
    ['model_version']
)

model_accuracy = Gauge(
    'model_accuracy',
    'Model accuracy on recent batch',
    ['model_version']
)

data_drift = Gauge(
    'data_drift_detected',
    'Whether data drift detected',
    ['feature']
)

# Middleware for metrics<a></a>
from fastapi import Request

@app.middleware("http")
async def add_process_time_header(request: Request, call_next):
    start_time = time.time()

    response = await call_next(request)

    process_time = time.time() - start_time
    response.headers["X-Process-Time"] = str(process_time)

    # Log metrics
    if request.url.path == "/predict":
        prediction_latency.labels(model_version="v1.2").observe(process_time)
        predictions_total.labels(model_version="v1.2", status="success").inc()

    return response

# Drift detection task<a></a>
from scipy.stats import ks_2samp

def monitor_drift(**context):
    """Monitor for data drift"""

    # Load training distribution
    train_data = pd.read_parquet('data/processed/features.parquet')

    # Load recent data
    recent_data = pd.read_parquet('data/raw/recent_batch.parquet')

    # Check KS statistic for numerical features
    for col in ['feature_1', 'feature_2', 'feature_3']:
        stat, p_value = ks_2samp(train_data[col], recent_data[col])
```

```

        if p_value < 0.01: # Significant drift
            data_drift.labels(feature=col).set(1)
            logger.warning(f"Drift detected in {col}: KS={stat:.4f}, p={p_value:.6f}")
        else:
            data_drift.labels(feature=col).set(0)

# Grafana dashboard (as code)<a></a>
grafana_dashboard = {
    "dashboard": {
        "title": "ML Pipeline Monitoring",
        "panels": [
            {
                "title": "Predictions Per Second",
                "targets": [
                    {"expr": "rate(predictions_total[5m])"}
                ]
            },
            {
                "title": "Prediction Latency (p95)",
                "targets": [
                    {"expr": "histogram_quantile(0.95, prediction_latency_seconds)"}
                ]
            },
            {
                "title": "Model Accuracy Trend",
                "targets": [
                    {"expr": "model_accuracy"}
                ]
            },
            {
                "title": "Data Drift Alert",
                "targets": [
                    {"expr": "data_drift_detected"}
                ]
            }
        ]
    }
}

# Alert rules<a></a>
alert_rules = ""
groups:
- name: ml_pipeline
  rules:
    - alert: ModelAccuracyLow
      expr: model_accuracy < 0.90
      for: 10m
      annotations:
        summary: "Model accuracy below 90%"
        action: "Trigger retraining"

    - alert: PredictionLatencyHigh
      expr: histogram_quantile(0.95, prediction_latency_seconds) > 0.5
      for: 5m
      annotations:
        summary: "Prediction p95 latency > 500ms"
        action: "Investigate model size, optimize inference"

    - alert: DataDriftDetected
      expr: data_drift_detected > 0
      for: 1h
      annotations:
        summary: "Significant data drift detected"

```

```

        action: "Trigger automated retraining"
    """

```

3.10 Automated Retraining Decision

```

# dags/automated_retraining.py<a></a>
from airflow.operators.trigger_dagrun import TriggerDagRunOperator

def should_retrain(**context):
    """Determine if retraining is needed"""

    client = mlflow.tracking.MlflowClient()

    # Check recent model performance
    runs = client.search_runs(
        experiment_ids=['ml_pipeline'],
        max_results=100
    )

    recent_runs = [r for r in runs if r.status == 'FINISHED'][-10:]

    if not recent_runs:
        return False

    # Get baseline
    baseline_auc = 0.92

    # Check if performance degraded
    recent_auc = recent_runs[-1].data.metrics.get('test_auc', 0)

    if recent_auc < baseline_auc * 0.95:
        logger.warning(f"Model degraded: {recent_auc:.4f}")
        return True

    # Check if data drift detected
    if context['task_instance'].xcom_pull(task_ids='monitor_drift', key='drift_detected'):
        logger.warning("Data drift detected")
        return True

    # Check monthly cadence
    import datetime
    last_training = recent_runs[-1].start_time
    if (datetime.datetime.now() - datetime.datetime.fromtimestamp(last_training)) >= datetime.timedelta(days=30):
        logger.info("Monthly retraining cadence")
        return True

    return False

with DAG(
    'ml_retraining_trigger',
    default_args=default_args,
    schedule_interval='0 6 * * *', # 6 AM daily check
    start_date=datetime(2024, 1, 1)
) as dag:

    check_retrain = PythonOperator(
        task_id='check_if_retrain',
        python_callable=should_retrain,
        provide_context=True
    )

    trigger_training = TriggerDagRunOperator(
        task_id='trigger_training_pipeline',

```

```

        trigger_dag_id='ml_training_pipeline',
        condition=lambda: check_retrain.result # Only if should_retrain returned True
    )

    check_retrain >>> trigger_training

```

3.11 Interview Talking Points for Project 3

Opening

> "I built a fully automated ML operations pipeline that orchestrates the entire ML lifecycle—from data ingestion to model deployment and monitoring. The system automatically retrains when performance degrades, validates data quality, tracks model lineage in MLflow, and alerts when issues occur. This demonstrates I can work in production ML environments."

Key Technologies Explained

Q: Explain your data validation approach.

> "I used Great Expectations to define a schema and validation rules for incoming data. Before training, the pipeline checks: (1) Do columns match expected types? (2) Are values in reasonable ranges? (3) Is the distribution similar to training data? If validation fails, we alert the team and skip training—preventing bad models from garbage data."

Q: Why did you use a feature store?

> "A feature store (I used DuckDB, but Feast is industry standard) serves two critical purposes: (1) Reproducibility—features computed in training must be identical to serving. (2) Performance—pre-computed features are cached and reused across experiments, avoiding recomputation. It also tracks feature lineage: which raw data produced which features."

Q: Walk me through your retraining trigger.

> "The pipeline runs a daily check for three conditions: (1) Has model accuracy dropped >5% from baseline? (2) Is data distribution significantly different (KL divergence)? (3) Has 30 days passed since last training? If any condition is true, Airflow automatically triggers retraining. This prevents silent model degradation."

Q: How do you prevent model regression in production?

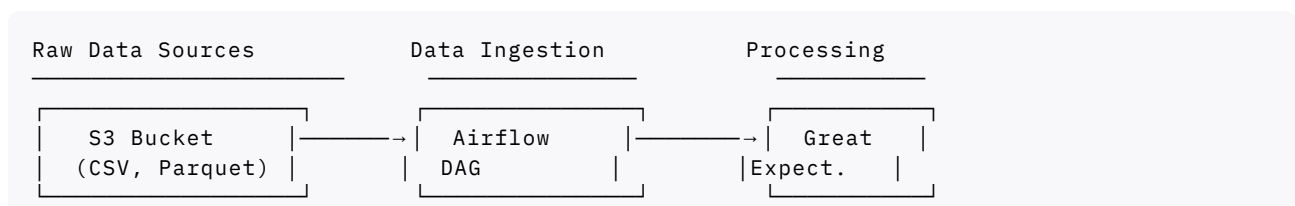
> "New models must score >95% of the old model's performance on test set before promotion. I also do canary deployment: route 10% traffic to the new model for 1 hour, monitor accuracy/latency, then either promote to 100% or rollback. This catches deployment issues early."

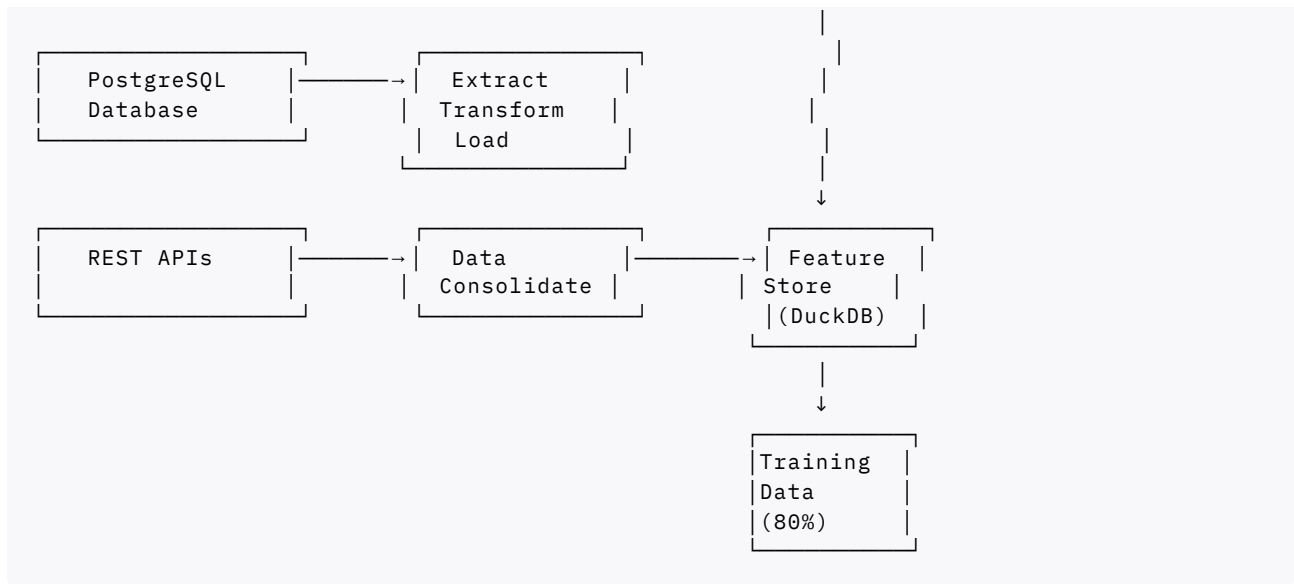
Q: Describe your monitoring architecture.

> "Prometheus collects real-time metrics: predictions/second, latency (p50/p95/p99), model accuracy on recent batches. Grafana visualizes these in dashboards. AlertManager triggers alerts (Slack/PagerDuty): if accuracy <90% or latency >500ms, we're notified. This early warning lets us act before customers notice degradation."

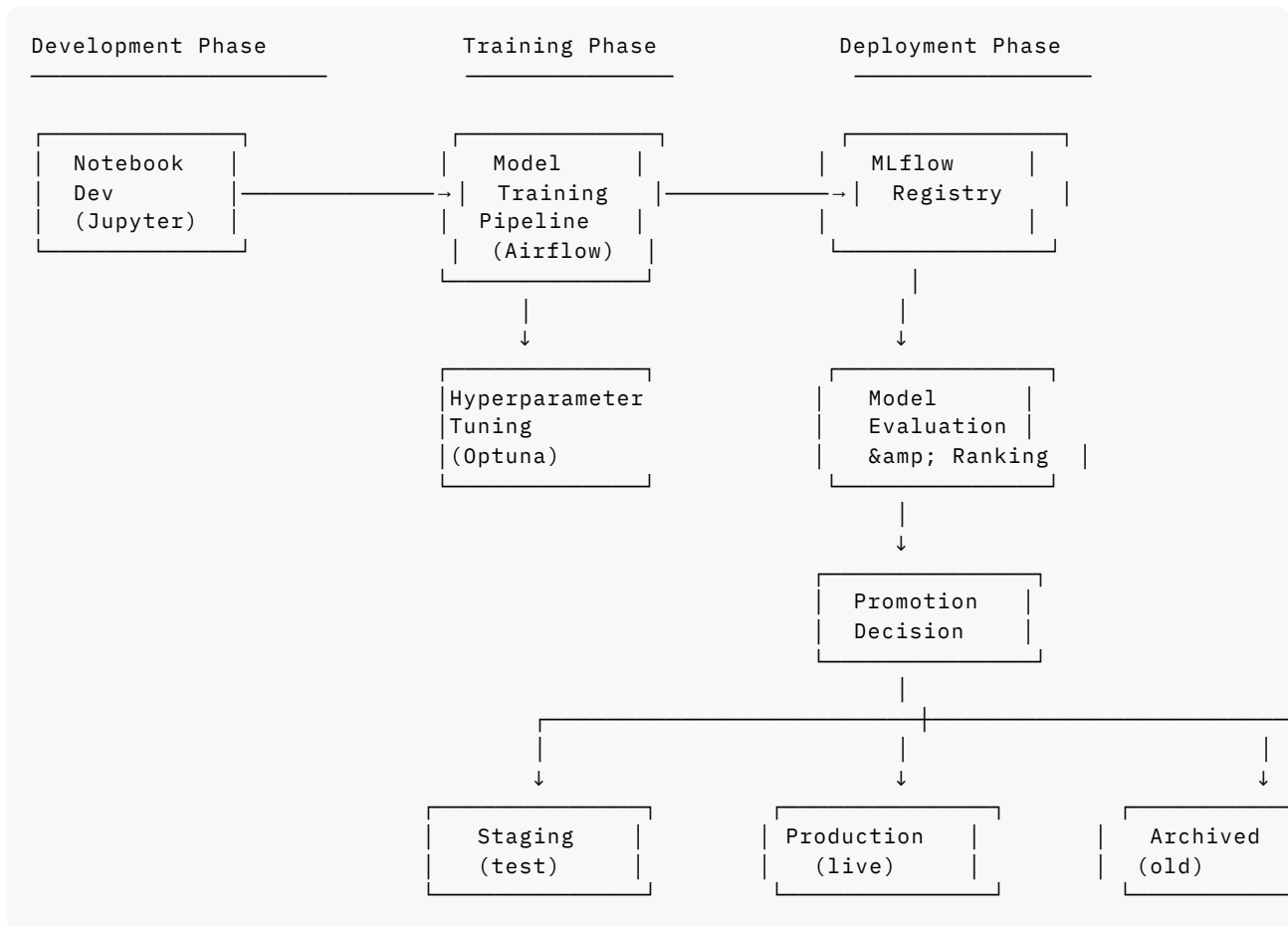
System Diagrams & Architecture

Data Flow Diagram





Model Lifecycle Diagram



Code Templates & Implementation

Complete Fraud Detection Training Pipeline

```
# pipelines/fraud_detection_complete.py<a></a>
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from imblearn.over_sampling import SMOTE
import xgboost as xgb
import mlflow
import joblib
import logging

logging.basicConfig(level=logging.INFO)
logger = logging.getLogger(__name__)

class FraudDetectionPipeline:
    def __init__(self, experiment_name="fraud_detection"):
        mlflow.set_experiment(experiment_name)
        self.scaler = StandardScaler()
        self.smote = SMOTE(random_state=42)

    def load_and_preprocess(self, data_path):
        """Load and preprocess data"""
        logger.info("Loading data...")
        df = pd.read_csv(data_path)

        # Handle missing values
        df = df.fillna(df.median(numeric_only=True))

        # Separate target
        X = df.drop('Class', axis=1)
        y = df['Class']

        logger.info(f"Dataset shape: {X.shape}, Fraud rate: {y.mean():.2%}")

        return X, y

    def train_and_evaluate(self, X, y):
        """Train model and evaluate"""

        # Split
        X_train, X_test, y_train, y_test = train_test_split(
            X, y, test_size=0.2, stratify=y, random_state=42
        )

        # Scale
        X_train_scaled = self.scaler.fit_transform(X_train)
        X_test_scaled = self.scaler.transform(X_test)

        # Balance training data
        X_train_balanced, y_train_balanced = self.smote.fit_resample(
            X_train_scaled, y_train
        )

        logger.info(f"After SMOTE: {y_train_balanced.value_counts().to_dict()}")

        # Train
        with mlflow.start_run(run_name="fraud_model"):
            model = xgb.XGBClassifier(
                n_estimators=200,
                max_depth=6,
                learning_rate=0.1,
                scale_pos_weight=len(y_train) / (y_train == 1).sum(),
                random_state=42,
```

```

        n_jobs=-1
    )

    model.fit(X_train_balanced, y_train_balanced)

    # Evaluate
    y_pred_proba = model.predict_proba(X_test_scaled)[: , 1]

    from sklearn.metrics import roc_auc_score, f1_score, precision_score, recall_score

    metrics = {
        'roc_auc': roc_auc_score(y_test, y_pred_proba),
        'f1': f1_score(y_test, model.predict(X_test_scaled)),
        'precision': precision_score(y_test, model.predict(X_test_scaled)),
        'recall': recall_score(y_test, model.predict(X_test_scaled))
    }

    # Log to MLflow
    mlflow.log_metrics(metrics)
    mlflow.xgboost.log_model(model, "fraud_model")

    logger.info(f"Model metrics: {metrics}")

    return model, X_test_scaled, y_test, metrics

if __name__ == "__main__":
    pipeline = FraudDetectionPipeline()
    X, y = pipeline.load_and_preprocess('creditcard.csv')
    model, X_test, y_test, metrics = pipeline.train_and_evaluate(X, y)

    logger.info("✔ Training complete!")

```

Testing & Production Considerations

Unit Tests

```

# tests/test_models.py
import pytest
import pandas as pd
import numpy as np
from sklearn.datasets import make_classification

@pytest.fixture
def sample_data():
    X, y = make_classification(n_samples=1000, n_features=20, random_state=42)
    return pd.DataFrame(X, columns=[f'feature_{i}' for i in range(20)], pd.Series(y))

def test_model_training(sample_data):
    """Test that model trains without errors"""
    X, y = sample_data

    model = train_model(X, y)

    assert model is not None
    assert hasattr(model, 'predict')

def test_prediction_shape(sample_data):
    """Test that predictions have correct shape"""
    X, y = sample_data
    X_train, X_test = X[:800], X[800:]

    model = train_model(X_train, y[:800])
    predictions = model.predict(X_test)

```

```

    assert len(predictions) == len(X_test)

def test_prediction_values(sample_data):
    """Test that predictions are valid probabilities"""
    X, y = sample_data
    X_train, X_test = X[:800], X[800:]

    model = train_model(X_train, y[:800])
    proba = model.predict_proba(X_test)

    assert np.all(proba >= 0)
    assert np.all(proba <= 1)
    assert np.allclose(proba.sum(axis=1), 1)

def test_api_endpoint():
    """Test API endpoint"""
    from fastapi.testclient import TestClient
    from app.main import app

    client = TestClient(app)
    response = client.post("/predict", json={
        "amount": 100,
        "hour": 12,
        # ... other features
    })

    assert response.status_code == 200
    assert "fraud_probability" in response.json()

```

Interview Talking Points

General Project Opening

> "I built three end-to-end ML projects that demonstrate production-grade engineering: (1) **Fraud Detection**—handling imbalanced data, explainability, real-time serving; (2) **Recommendations**—collaborative + content-based filtering, hybrid scoring; (3) **MLOps**—automated pipelines, monitoring, retraining. Together, these show I can build, deploy, and maintain ML systems at scale."

Technical Depth Questions

Q: What's the biggest challenge you faced?

> "Imbalanced data in fraud detection. Using raw class weighting degraded recall too much. SMOTE balanced the training set, but I had to apply it only to training (not test) to avoid data leakage. I also used Precision-Recall curves instead of ROC-AUC for evaluation—better for imbalanced problems."

Q: How do you ensure your model doesn't degrade in production?

> "Three layers: (1) **Testing**: validate predictions on held-out test set before deployment. (2) **Monitoring**: track model accuracy on recent batches using Prometheus/Grafana. (3) **Automated retraining**: when accuracy drops >5% or data drift is detected, automatically retrain and promote if metrics improve."

Q: Explain your recommendation system.

> "Hybrid approach combining collaborative (what similar users liked) and content-based (what's similar to items you liked). I used matrix factorization (ALS) for collaborative, TF-IDF for content, then weighted both signals (60% collab, 40% content). Added diversity constraints to avoid recommending only one genre."

Appendix

Resources

Learning

- **MLOps Foundations:** deeplearning.ai/machine-learning-engineering-for-production
- **Fast.ai Practical Deep Learning:** fast.ai
- **MLOps.community:** mlops.community

Tools Documentation

- **MLflow:** mlflow.org
- **Airflow:** airflow.apache.org
- **FastAPI:** fastapi.tiangolo.com
- **Great Expectations:** docs.greatexpectations.io

Datasets

- **Kaggle Credit Card Fraud:** kaggle.com/mlg-ulb/creditcardfraud
- **MovieLens:** grouplens.org/datasets/movielens
- **UCI ML Repository:** archive.ics.uci.edu

Useful Commands

Docker

```
# Build<a></a>
docker build -t ml-fraud-api .

# Run<a></a>
docker run -p 8000:8000 ml-fraud-api

# Push to registry<a></a>
docker tag ml-fraud-api myregistry/ml-fraud-api:v1.0
docker push myregistry/ml-fraud-api:v1.0
```

Kubernetes

```
# Deploy<a></a>
kubectl apply -f k8s/deployment.yaml

# Check status<a></a>
kubectl get pods
kubectl logs <pod-name>

# Scale<a></a>
kubectl scale deployment ml-api --replicas=3
```

MLflow

```
# Start server<a></a>
mlflow server --backend-store-uri postgresql://user:pwd@localhost/mlflow --default-artifact-

# View UI<a></a>
# Navigate to http://localhost:5000<a></a>
```

Airflow

```
# Initialize<a></a>
airflow db init

# Create user<a></a>
airflow users create --username admin --password admin

# Start scheduler<a></a>
airflow scheduler

# Start webserver<a></a>
airflow webserver --port 8080
```

Common Errors & Solutions

Error	Cause	Solution
Data leakage	Applied SMOTE to entire dataset	Apply SMOTE only to training set
Model doesn't improve	Learning rate too high	Reduce learning_rate, increase iterations
API too slow	Loading model on every request	Load model once at startup
Training fails	Out of memory	Use data batching or reduce batch_size
Predictions are constant	Model didn't learn properly	Check data preprocessing, feature importance

This handbook is your complete guide to production ML. Build, deploy, and iterate! 📖